# LINUX

2017-2018

## Présentation du système

Ce support est destiné aux étudiants de BTS SIO

Pré-requis : Aucun

Version linux utilisée : Debian 9.01 32 bits avec le bureau Mate

# Linux

## PRESENTATION DU SYSTEME

## INTRODUCTION.

Linux est un système d'exploitation de type UNIX, multitâches et multi-utilisateurs pour machines à processeurs 32 et 64 bits ouvert sur les réseaux et les autres systèmes d'exploitation.

La principale singularité de Linux est d'être un logiciel libre, développé de façon collaborative et pour une grande part bénévole par des milliers de programmeurs répartis dans le monde.

Ce modèle de développement joue un grand rôle dans la qualité du résultat obtenu, qui est considéré par des analystes indépendants comme très supérieurs à des systèmes commerciaux similaires, par exemple Windows NT (2000, 2003, 2008…).

### Avantages de Linux.

Linux est un système :

- **Puissant.** Il permet de faire faire beaucoup de choses à sa machine avec un minimum de ressources.
- **Efficace.** Contrairement à des systèmes beaucoup plus répandus, il n'utilise pour ses besoins propres que très peu de ressources. Les logiciels que vous utilisez pour votre travail et les services que vous mettez en place, disposent donc de beaucoup plus de puissance pour fonctionner. Le système peut paraître moins « confortable » qu'un autre, mais c'est parce qu'il dédit un maximum de puissance aux tâches principales, se souciant moins de l'utilisateur. De fait, l'utilisateur doit compenser un peu ce manque de confort par des compétences plus élevées.
- **Fiable.** Une machine sous Linux connaît peu de bug. La communauté « linuxienne » se charge sans cesse de la correction de ceux-ci. Le très grand nombre d'utilisateurs, dans des situations diverses, utilisant des logiciels « métiers » très divers, permet d'avoir une plus grande maîtrise des failles inhérentes à tout logiciel.
- **Robuste.** C'est un système simple et très modulaire ; Chaque partie de l'OS ou des services est faiblement dépendante, voire pas du tout, du service voisin. Cette faible dépendance permet de développer de manière fiable et stable une nouvelle fonctionnalité.
- **Très bon marché.** Le prix demandé par les sociétés qui vendent Linux sur CDROM ne sert qu'à couvrir leurs frais et à leur permettre de financer dans une certaine mesure la poursuite de cette activité. Linux étant développé par des passionnés pour le plaisir, personne n'a à supporter le coût de son développement. Néanmoins, les logiciels « métiers » peuvent être vendus.

- Enfin, Linux est conforme à la norme **POSIX** et aux standards du marché, en particulier de l'Internet. Cela signifie que qu'un logiciel conçu pour un autre système de la même famille (Solaris de SUN, Digital Unix, AIX d'IBM, SCO Unix...) peut être rapidement porté sous Linux et vice-versa, ce qui assure une protection de l'investissement logiciel en cas d'obligation de changement de système.

Comme on le voit, Linux est un système donnant satisfaction aussi bien sur de des machines anciennes ou bas de gamme que sur des machines puissantes très sollicitées ou devant remplir des fonctions importantes.

## Principales utilisations de Linux

- Comme **serveur de fichiers et d'impression :** Linux supporte les trois principaux protocoles de partages de fichiers : NFS pour clients UNIX, SMB pour clients Windows et AppleShare pour clients Osx, ainsi que les protocoles de partages d'imprimantes. Linux peut également servir de serveur de fax.

- Comme **serveur Internet/Intranet** : On trouve dans les distributions standards de Linux tous les logiciels nécessaires pour réaliser un serveur Internet complet, même sur des machines de puissance modeste. Cela inclut des fonctionnalités :
    - De transfert et de distribution du courrier électronique, des *news* (Usenet).
    - De serveur Web ou FTP.
    - De serveur de noms de machines ou de domaines (DNS, DDNS,…)

- Comme **serveur d'applications client/serveur** : un grand nombre de logiciels de serveurs de bases de données (SGBD), relationnels, relationnels-objets ou objets, commerciaux ou libres, sont disponibles pour Linux. Avec ces logiciels, surtout du côté commercial, ce sont des milliers d'applicatifs

- Comme **station de développement** : Linux dispose, le plus souvent sous forme de logiciels libres, des outils de développement pour la plupart des langages actuels : C, C++, Fortran, Java, Cobol, LISP, Prolog, SmallTalk. On trouve aussi des outils pour le contrôle des sources, pour le travail en groupe, pour le suivi des erreurs, pour le testage.

- Comme **station bureautique** : grâce à des suites bureautiques intégrées commerciales comme Applix ou StarOffice et à des environnements graphiques comme KDE ou fvwm95, les stations bureautiques sous Linux offre les même fonctionnalités que leurs équivalents sous Windows ou Osx: traitement de texte, tableur, logiciel de présentation et de dessin, gestion de fichiers, partage de documents, intégration des technologies Internet/Intranet (courrier électronique, Web).

- Comme **station réseau** : Linux permet d'accéder aux services les plus populaires de l'Internet : Web, FTP (transferts de fichiers), news (messageries thématiques), courrier électronique, IRC (discussions en temps réel).

- Comme routeur, serveur vocal, serveur de monitoring, pare-feu, proxy, serveur VPN, d'authentification…

## Questions généralement posées.

### Il paraît que Linux est gratuit. C'est vrai ?

Oui et non.

Linux est gratuit si on le télécharge par Internet ou si on le recopie depuis un CD-ROM prêté par un ami, ce qui, contrairement aux logiciels commerciaux, est légal et même recommandé.

Il existe cependant des distributions commerciales de Linux, qui incluent des logiciels commerciaux avec des licences restrictives et qui ne sont donc pas librement recopiables telles quelles.

**Existe-t-il d'autres systèmes d'exploitation libres comparables à Linux ?**

Oui. Les systèmes dérivés de BSD, notamment FreeBSD, NetBSD et OpenBSD, sont également des systèmes d'exploitations libres. Leurs licences d'utilisations, leurs modèles de développements et leurs modes de diffusion sont cependant différents de ceux de Linux.

**Combien de personnes utilisent Linux dans le monde ?**

On ne peut pas donner de chiffre précis, mais début 1998 plusieurs estimations évaluaient ce nombre entre 4 et 8 millions, ce qui en fait le premier système d'exploitation d'origine non-Microsoft. La croissance de Linux sur ces dernières années a été de l'ordre de 80 à 100% par an, malgré une couverture à peu près nulle par les media.

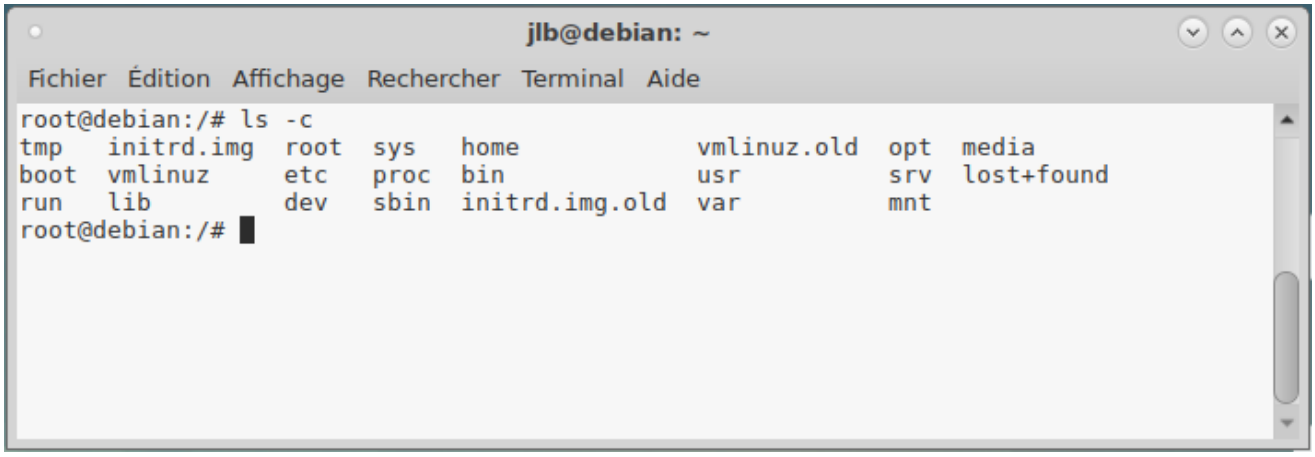
Aujourd'hui il est difficile de dire combien d'utilisateurs il y a. On estime néanmoins qu'il y a 25 millions d'utilisateurs seulement pour Ubuntu dans le monde.

# PRESENTATION DE LINUX

**Rappel : les manipulations ont été réalisées avec Debian 9.01 en version 32 bits le bureau utilisé et Mate**

## Le système de gestion de fichiers

Le système de fichiers racine (root file system), soit le système de fichiers primaire est associé au répertoire le plus haut « / » (on dit souvent « racine ») :




| | |
|---|---|
| /bin | **commandes binaires (exécutables) utilisateur essentielles (pour tous les utilisateurs)** |
| /boot | **fichiers statiques du chargeur de lancement (fichiers de lancement de Linux)** |
| /dev | **fichiers de périphériques (chaque périphérique –** device **– est associé à un dossier. Ex : /dev/cdrom représente le lecteur de cdrom/dvdrom/bdrom).** |
| /etc | **configuration système spécifique à la machine** |
| /home | **répertoires personnels des utilisateurs (optionnel)** |
| /lib | **bibliothèques partagées essentielles et modules du noyau** |

| |
|---|
| /mnt **point de montage pour les systèmes de fichiers montés temporairement** /proc **système de fichiers virtuel d'information du noyau et des processus** /root **répertoire personnel de root (optionnel)** |
| /sbin **binaires système (binaires auparavant mis dans /etc)** |
| /sys **état des périphériques (model device) et sous-systèmes (subsystems)** /tmp **fichiers temporaires** |

## CD

## COMMANDES DE BASES LINUX

Le but de cette partie est la prise en main des commandes de base de l'environnement Linux.

**1. Introduction**

- Qu'est-ce que le **shell** ?

C'est l'interpréteur de commandes (l'interface) entre l'utilisateur et le système d'exploitation, d'où son nom anglais «shell», qui signifie «coquille».

Le shell est ainsi chargé de faire l'intermédiaire entre le système d'exploitation et l'utilisateur grâce aux lignes de commandes saisies par ce dernier. Son rôle consiste ainsi à lire la ligne de commande, interpréter sa signification, exécuter la commande, puis retourner le résultat sur les sorties.

Il existe plusieurs types de shells, les plus connus depuis Unix ayant une version améliorée sous Linux. Le fichier /etc/shells contient une liste de tous les shells disponibles :

- /bin/**ash**
- /bin/**bash**
- /bin/**bash1** /bin/**csh**
- /bin/false
- /bin/passwd /bin/**sh**
- /bin/**tcsh**
- /usr/bin/**csh** /usr/bin/**ksh** /usr/bin/**tcsh** /usr/bin/**zsh**

Les plus connus sont bash (version améliorée du shell Bourne sous Unix), ksh (version améliorée du shell Korn sous Unix) et tcsh (version améliorée du Shell C sous Unix). La commande help affiche la liste des commandes internes du shell. Par défaut, c'est le shell Bash qui est installé avec Linux. C'est aussi le plus puissant et le plus utilisé, c'est pourquoi c'est celui-ci qui sera utilisé dans les sections suivantes.

Chaque utilisateur possède un shell par défaut, qui sera lancé à l'ouverture d'une invite de commande (le Terminal). Le shell par défaut est précisé dans le fichier de configuration **/etc/passwd** dans le dernier champ de la ligne correspondant à l'utilisateur. Il est possible de changer de shell dans une session en exécutant tout simplement le fichier exécutable correspondant, par exemple : **/bin/bash**

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
  GNU nano 2.7.4                    Fichier : passwd

gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-timesync:x:100:102:systemd Time Synchronization,,,:/run/systemd:/bin/false
systemd-network:x:101:103:systemd Network Management,,,:/run/systemd/netif:/bin/false
systemd-resolve:x:102:104:systemd Resolver,,,:/run/systemd/resolve:/bin/false
systemd-bus-proxy:x:103:105:systemd Bus Proxy,,,:/run/systemd:/bin/false
_apt:x:104:65534::/nonexistent:/bin/false
rtkit:x:105:109:RealtimeKit,,,:/proc:/bin/false
dnsmasq:x:106:65534:dnsmasq,,,:/var/lib/misc:/bin/false
messagebus:x:107:110::/var/run/dbus:/bin/false
usbmux:x:108:46:usbmux daemon,,,:/var/lib/usbmux:/bin/false
speech-dispatcher:x:109:29:Speech Dispatcher,,,:/var/run/speech-dispatcher:/bin/false
lightdm:x:110:111:Light Display Manager:/var/lib/lightdm:/bin/false
pulse:x:111:112:PulseAudio daemon,,,:/var/run/pulse:/bin/false
avahi:x:112:114:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/bin/false
jlb:x:1000:1000:jlb,,,:/home/jlb:/bin/bash
```

Pour vérifier le contenu de votre fichier passwd il vous faut :

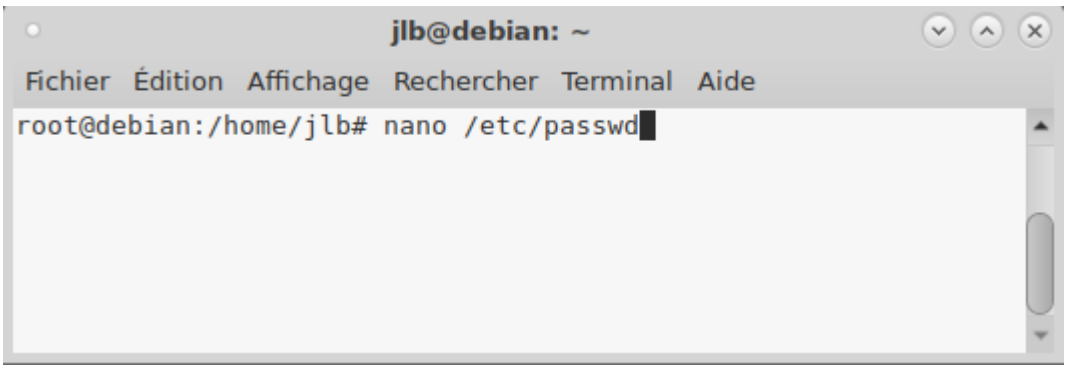
- Lancer le terminal (Application -> Outils sytèmes -> Terminal Mate

Le terminal s'ouvre comme celui-ci :

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
jlb@debian:~$
```

Vous voyez le signe $ ? Il signifie que vous avez les droits utilisateurs. Toutes les manipulations que vous ferez dans ce mode ne remettront pas en cause l'intégrité du système. Pour pouvoir passer en Super Utilisateur il faut saisir **su**, entrer le mot de passe du Super Utilisateur (attention la frappe est invisible, vous n'avez aucun écho chaque fois que vous tapez une touche) et le signe # apparait.

Maintenant saisissez nano /etc/password




Validez, vous venez de lancer l'éditeur de fichier en mode Super Utilisateur et ouvrir le fichier passwd.

**Pour quitter Nano appuyer sur la touche Ctrl + la touche X**

## Commandes pour débuter

Avant de commencer, il faut savoir que Linux est **sensible à la casse** (*case sensitive* en anglais), c'est à dire qu'il distingue les majuscules des minuscules. Ainsi, pour créer un répertoire, la commande est 'mkdir', ce n'est pas la peine d'essayer MKDIR ou mKdiR, cela ne fonctionnera pas. De même, les noms de fichiers et de répertoires sont également sensibles à la casse.

De plus, sous Unix, les chemins sont séparés par des « slash » : **/etc** ou **/init/xfs** mais jamais **etc\init\xfs**.

**Répertoires spéciaux :**

**.** Représente le répertoire courant,

**..** Représente le répertoire parent,

~ représente le répertoire maison (home) de l'utilisateur.

***Fichiers cachés*** *:*

Sous Unix, les fichiers cachés commencent par un point. Par exemple, ~/.bashrc est un fichier caché, dans le répertoire maison de l'utilisateur, qui contient la configuration de son shell.

***Jokers*** *:* ***?*** *et* *

Les caractères ? et * dans les noms de fichiers et de répertoires permettent de représenter des caractères quelconques. '?' représente un seul caractère, tandis que '*' en représente un nombre quelconque.

Par exemple "*.jpg" représente tous les fichiers se terminant par jpg ; "*toto*" tous les fichiers contenant "toto".

**Ex** : Testez dans /etc : **ls mt*** et **ls *mt***

[**nota :** Allez dans un dossier « cd », remonter au dossier parent « cd .. », remonter à la racine « cd / »]

Il faut également savoir que c'est le shell qui interprète ces caractères avant de transmettre la ligne de commande. Par exemple, si vous tapez : rm f *.tmp, le shell transformera cette ligne de commande en : rm truc 1.tmp truc2.tmp truc3.tmp.

## Commandes

Une commande est l'exécution d'un programme dans l'interprète (**Shell**). Elle prend en entrée des options et/ou des paramètres. Elle peut renvoyer de l'information à l'écran ou dans un fichier, modifier un fichier, ou produire un message d'erreur.

Une description de toutes les commandes est disponible avec la commande **man** ou **help**. N'hésitez pas à l'utiliser.

Voici les commandes de base sous Linux :

| Commandes linux | équivalent MsDos | à quoi ça sert | Exemples : |
|---|---|---|---|
| cd | cd | change le répertoire courant. | cd ..<br>- va dans le répertoire parent du répertoire courant<br><br>cd /home/user/.nsmail<br>- va dans le répertoire désigné |
| ls | dir | affiche le contenu d'un répertoire | ls<br>- affiche le contenu du répertoire courant<br><br>ls -l<br>- affiche le contenu du répertoire courant de manière détaillée<br><br>ls -a /home/user<br>- affiche le contenu du répertoire désigné (ainsi que les fichiers cachés) |
| cp | copy<br>xcopy | copie un ou plusieurs fichiers | cp toto /tmp<br>- copie le fichier toto dans le répertoire /tmp<br><br>cp toto titi<br>- copie le fichier toto sur le fichier titi<br><br>cp -R /home/user /tmp/bak<br>- copie le répertoire /home/user ainsi que tout ce qu'il contient dans /tmp/bak |
| rm | del | efface un ou plusieurs fichiers | rm toto titi<br>- efface les fichiers toto et titi<br><br>rm -f toto titi<br>- efface les fichiers toto et titi sans demander confirmation |
| rm -rf | deltree | efface un répertoire et son contenu | rm -rf /tmp/*<br>- efface (sans demander de confirmation) tous les fichiers et répertoire de /tmp |
| mkdir | md | crée un répertoire | mkdir /home/user/mes documents<br>- crée le répertoire "mes documents" dans le sous répertoire /home/user |
| rmdir | rm | efface un répertoire s'il est vide | rmdir /home/user/.nsmail<br>- efface le répertoire .nsmail de /home/user si celui-ci est vide |
| mv | ren<br>move | déplace ou renomme un ou des fichiers | mv tata titi<br>- renomme tata en titi<br><br>mv * *.bak<br>- ne fonctionne pas !!!!<br><br>mv * /tmp/bak<br>- déplace tous les fichiers du répertoire courant vers le répertoire /tmp/bak |

| | | | |
|---|---|---|---|
| **chown** | pas d'équivalent | modifie le propriétaire d'un fichier | chown user unfichier<br>rend user propriétaire de unfichier. |
| **chgrp** | pas d'équivalent | modifie le groupe proprétaire d'un fichier | chgrp -R nobody /home/httpd<br>- rend le groupe : nobody (un groupe ayant très peu de droit sur un système linux) propriétaire de /home/httpd ainsi que tout les fichiers qu'il contient (-R) |
| **ln -s** | pas d'équivalent | crée un lien vers un fichier | ln -s /dev/fd0 /dev/disquette<br>crée un lien vers /dev/fd0 (le lecteur de disquette) nommé /dev/disquette. La manipulation de /dev/fd0 et /dev/disquette (sauf l'effacement). |
| **grep** | pas d'équivalent | recherche une chaine dans un fichier (en fait recherche une expression régulière dans plusieurs fichiers) | grep chaine *.txt<br>- recherche la chaine 'chaine' dans tous les fichier se terminant par .txt. |
| **which** | pas d'équivalent | trouve le répertoire dans lequel se trouve une commande | which emacs<br>- retourne le nom du répertoire dans lequel se trouve la commande emacs. |
| **cat** | **type** | affiche un fichier à l'écran | cat ~/.bashrc<br>- affiche le contenu du fichier ~/.bashrc |
| **mv** | **ren**<br>**move** | déplace ou renomme un ou des fichiers | mv tata titi<br>- renomme tata en titi<br><br>mv * *.bak<br>- ne fonctionne pas !!!!<br><br>mv * /tmp/bak<br>- déplace tous les fichiers du répertoire courant vers le répertoire /tmp/bak |
| **find** | **dir -s** | trouve un fichier répondant à certains critères | find /home -name "*bash*"<br>- trouve tous les fichiers contenant le mot bash dans leur nom se trouvant dans le répertoire /home |
| **locate** | **dir -s** | trouve un fichier d'après son nom | locate bash<br>- trouve tous les fichiers contenant le mot bash dans leur nom complet (avec le répertoire) : à la différence de find, locate trouve ses informations dans une base de donnée créée par updatedb |
| **man** | **help** | affiche l'aide concernant une commande particulière | man ls<br>- affiche l'aide (page de manuel) de la commnade ls. On quitte man en appuyant sur la touche 'q' |
| **chmod** | pas d'équivalent | modifie les permissions d'un fichier | chmod o+r /home/user<br>- autorise les autres (o=other) (ie: ceux qui ne sont ni le propriétaire, ni membre du groupe propriétaire) à lire (r=read) le répertoire /home/user<br><br>chmod a+rw /home/user/unfichier<br>- autorise tout le monde (a=all) à lire et écrire (w=write) dans le fichier /home/user/unfichier |

## Aliases

Plutôt que de taper de longues commandes, ou bien parce que vous préférez vous rappeler d'un nom plutôt que du vrai nom Unix, vous pouvez définir des aliases. Pour ce faire, utilisez la commande **alias** comme suit :

Si votre shell est bash ou sh ou ash (par défaut) :

alias md=mkdir

alias ls='ls --color' (pratique, les dossiers sont identifiés en bleu lorsque vous êtes administrateur !)

alias eclip2='telnet eclip2.ec-lille.fr'

Si votre shell est tcsh ou csh (par défaut) : alias md mkdir

alias ls 'ls --color'

alias eclip2 'telnet eclip2.ec-lille.fr'

Ainsi pourrez-vous taper md au lieu de mkdir, et eclip2 pour vous connecter à cette machine via telnet ; la commande ls affichera une sortie en couleur…

Les alias ne sont pas permanents. Pour pouvoir les enregistrer rajoutez la ligne d'alias dans le fichier **.bashrc**.

# LE SYSTEME DE FICHIER UNIX

## La commande ls

Cette commande est omniprésente, aussi il est bon d'en présenter les basiques.

Afficher le listing page par page :

**ls | less (less est une version améliorée de more)**
« | » est le caractère « tube », il est effectué avec AltGr+ '6' ou Ctrl+Alt+'6'

Afficher le listing en couleurs :

**ls --color**

Afficher aussi les fichiers cachés (commençant par un point) :

**ls -a**

Mettre un '/' après les noms de répertoires :

**ls -p**

Afficher le listing détaillé :

**ls –l** (lettre « L » minuscule)

**Tri sur la date**

Pour afficher les fichiers d'un répertoire en triant sur la date de mise à jour des fichiers Afficher les fichiers les plus récents en premier :

**ls -t**

Afficher les fichiers les plus vieux en premier :

**ls -rt**

Mixer avec l'option "l" afin d'afficher le listing détaillé : ls -rtl ou ls -tl

Bien sûr, toutes ces options sont mixables, ainsi "ls -altp" affiche tous les fichiers, de façon détaillée, dans l'ordre chronologique, en ajoutant '/' après chaque nom de répertoire.

**Exemple de listing**

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
jlb@debian:~$ ls -altp
total 120
-rw-------  1 jlb  jlb  27974 nov.   3 11:27 .xsession-errors
-rw-------  1 jlb  jlb     42 nov.   3 11:27 .bash_history
drwxr-xr-x 14 jlb  jlb   4096 nov.   3 11:01 ./
-rw-------  1 jlb  jlb    628 nov.   3 11:00 .ICEauthority
-rw-------  1 jlb  jlb     51 nov.   3 11:00 .Xauthority
-rw-------  1 jlb  jlb   3845 nov.   3 10:59 .xsession-errors.old
drwx------  3 jlb  jlb   4096 nov.   3 10:56 .cache/
drwx------  5 jlb  jlb   4096 nov.   3 10:56 .config/
drwx------  3 jlb  jlb   4096 nov.   3 10:56 .local/
drwx------  3 jlb  jlb   4096 nov.   3 10:56 .gnupg/
drwxr-xr-x  2 jlb  jlb   4096 nov.   3 10:56 Bureau/
drwxr-xr-x  2 jlb  jlb   4096 nov.   3 10:56 Documents/
drwxr-xr-x  2 jlb  jlb   4096 nov.   3 10:56 Images/
drwxr-xr-x  2 jlb  jlb   4096 nov.   3 10:56 Modèles/
drwxr-xr-x  2 jlb  jlb   4096 nov.   3 10:56 Musique/
drwxr-xr-x  2 jlb  jlb   4096 nov.   3 10:56 Public/
drwxr-xr-x  2 jlb  jlb   4096 nov.   3 10:56 Téléchargements/
drwxr-xr-x  2 jlb  jlb   4096 nov.   3 10:56 Vidéos/
-rw-r--r--  1 jlb  jlb     55 nov.   3 10:56 .dmrc
-rw-r--r--  1 jlb  jlb    675 nov.   3 10:55 .profile
-rw-r--r--  1 jlb  jlb    220 nov.   3 10:55 .bash_logout
-rw-r--r--  1 jlb  jlb   3526 nov.   3 10:55 .bashrc
drwxr-xr-x  3 root root  4096 nov.   3 10:55 ../
jlb@debian:~$
```

**Explication :**

La première ligne "total 120" est l'espace disque utilisé par l'ensemble des fichiers du répertoire.

La première colonne -rw-r--r--représente les permissions associées au fichier. Le premier caractère est un **tiret** pour un fichier, un **d** pour un répertoire, un **l** pour un lien, etc. ensuite, on a trois groupes de trois caractères : rw- ou r-- ou rwx ou...

Le premier groupe représente les permissions associées à **l'utilisateur** (ici, jlb), le deuxième celles associées à son **groupe** (ici : jlb), enfin le dernier est les permissions que **tout le monde** a sur ces fichiers.

**r** signifie : possibilité de lire ce fichier / dans ce répertoire,

**w** signifie : possibilité d'écrire dans ce fichier / répertoire,

**x** signifie : possibilité d'exécuter ce fichier / d'aller dans ce répertoire.

La deuxième colonne présente le nombre d'inodes (partie élémentaire de système de fichiers) qui pointent vers le fichier/répertoire (généralement 1 pour un fichier, 2+le nombre de sous-répertoires pour un répertoire). Cette notion est vue plus loin.

La troisième colonne représente l'utilisateur à qui appartient le fichier.

La quatrième représente le groupe auquel le fichier appartient (users).

La cinquième la taille en octets.

La sixième la date et heure de modification.

La septième le nom du fichier/répertoire.

## Exercice :

Dans les systèmes d'exploitation dérives d'Unix, le codage des droits se fait sur 9 bits groupes par 3 bits. Ces droits sont codes en un entier. Pour ce faire, on convient de la correspondance : r = 4; w = 2 et x = 1.

Ainsi, les droits rw-, correspondent à l'entier (r=)4+(w=)2=6. Donc rw-rw-rw- correspond à l'entier 666.

Questions :

1. À quels droits correspondent les entiers 751; 521; 214 et 150 ?
2. Par quels entiers sont codés les droits rw-r- -r- - et rwxr-xr-x ?

# Éditeurs de texte

Un éditeur de texte permet de rentrer du texte dans un fichier afin de le conserver.

## L'Éditeur vi

vi (prononcez vie-aïe ou [vi:ai]) est l'éditeur de texte de base sous Linux, vous risquez bien d'avoir à vous en servir au plus mauvais moment, c'est à dire lorsque plus rien d'autre ne fonctionne.

### Lancer VI

Si vous tentez d'ouvrir un fichier inexistant, vi créera ce fichier. Vous pouvez donc créer un nouveau fichier simplement en tapant [ vi nom_du_fichier].

### Les modes de VI

vi possède deux modes : le mode "**Insert**" et le mode **normal**. En mode normal vous ne pouvez pas insérer de texte dans le fichier, mais les touches du clavier sont autant de touches de commandes. En mode Insert, les touches de commandes (notamment les lettres !) se transforment en vraies lettres que vous pouvez insérer dans le fichier.

### Insérer du texte

Lorsque vi s'ouvre, il est en mode normal. Pour passer en mode Insert :

Tapez [i] ou [Insert] pour insérer du texte à l'endroit où se trouve le curseur,

tapez [A] pour ajouter du texte à la fin d'une ligne.

En mode Insert, vous pouvez taper du texte, effacer avec la touche [Suppr] ou [Bkspace]. Pour quitter le mode Insert, tapez [Esc].

***Remarque :*** à la suite de votre fichier, vous voyez des lignes vides commençant par le caractère '—'. C'est normal : cela signifie juste que ces lignes sont vides, et les caractères '—' ne seront bien sûr pas enregistrés dans votre fichier.

### Les commandes

Après avoir quitté le mode Insert, ou avant d'y être entré, les touches du clavier correspondent à des commandes. Voici ci-dessous les commandes de base qui vous permettront de vous y retrouver (attention le caractère « : » est à taper) :

> **:q** [Entrée] pour quitter sans sauver ou parfois **:q!** (si vous ne voulez pas sauvegarder les modifications),
>
> **:w** [Entrée] pour enregistrer,
>
> **:wq** [Entrée] pour enregistrer et quitter,
>
> **x** efface le caractère qui se trouve sous le curseur,
>
> **dd** efface la ligne sur laquelle se trouve le curseur,
>
> **:u**[Entrée] permet d'annuler (ou **:undo**).

## L'Éditeur « gedit »

Gedit est l'éditeur de texte officiel de l'environnement graphique GNOME, par défaut sur Debian et Ubuntu. Pour les utilisateurs de l'interface KDE (KUbuntu) son équivalent est Kate. Ce logiciel, sous licence GPL,

propose une interface simple et facile d'utilisation, développée avec l'aide de la bibliothèque graphiques GTK.

Gedit permet l'édition simple et l'enregistrement d'un fichier texte (.txt) mais dispose aussi du support intégral de l'encodage de caractères Unicode (tous les caractères internationaux), de modes de surbrillance configurables pour de nombreux langages de programmation, de la vérification orthographique, d'un outil de recherche et de remplacements, de la numérotation des lignes, de l'indentation automatique pratique pour la programmation et de bien d'autres choses… que vi et Emacs ne font pas… mais il faut une interface graphique. Pour le lancer en mode administrateur, et modifier des fichiers systèmes, on le lance depuis le terminal administrateur ou avec la commande « sudo gedit » sous Ubuntu, sinon on le trouve dans le menu « Accessoires » de Debian.

Il crée automatiquement à l'enregistrement un fichier caché (fichier.txt~) de sauvegarde au cas où il vous faudrait récupérer une version antérieure. Cette fonction peut être désactivée.

Il possède des greffons (plugins) donnant accès à des fonctionnalités allant du sympathique (changement de la casse, indentation de lignes, etc.) au très pratique (terminal intégré, liste des balises, sélection de couleur, etc).

**Installation**

Gedit est l'éditeur de texte d'Ubuntu installé par défaut. Si vous ne l'avez pas, installez le paquet gedit.

**Utilisation**

Vous trouverez les options dans le menu Édition → Préférences.

**Par exemple :**

Pour empêcher la création de fichier de sauvegarde de la forme <monfichier>~ décochez la case Créer une copie de sauvegarde des fichiers avant l'enregistrement dans Éditeur → Enregistrement du fichier.

Vous pouvez mettre en évidence la parenthèse / l'accolade / autre correspondant à celle où se trouve votre curseur.

Un système d'indentation automatique est intégré, et peut être complété par des greffons (voir plus bas).

**Greffons**

Vous pouvez utiliser les greffons déjà fournis avec Gedit mais aussi en rajouter d'autres (en). Pour activer un greffon, allez dans Édition → Préférences → onglet Greffons. Tous les greffons disponibles sont affichés, vous n'avez qu'à cocher pour faire le choix.

Le paquet gedit-plugins ajoute de nombreuses possibilités notamment l'intégration d'une console dans le panneau inférieur.

**Gestion des onglets**

Vous pouvez utiliser le glisser-déposer pour organiser vos onglets comme vous le souhaitez.
Vous pouvez glisser un onglet hors de la fenêtre pour créer une nouvelle fenêtre. Vous pouvez aussi déplacer les onglets entre 2 fenêtres de l'éditeur.

**Jeux de couleur**

Gedit peut appliquer des jeux de couleurs à un code source donné, pour le rendre plus agréable à lire. Il existe cinq thèmes de base, mais vous pouvez en importer d'autres. Pour ça, allez dans Édition → Préférences → Police et Couleurs. Choisissez Ajouter pour ajouter un nouveau jeu de couleurs (un fichier .xml).

**Installation du dictionnaire français**

Pour installer le dictionnaire français, pour la vérification orthographique, il vous suffit d'installer le paquet aspell-fr… si ce n'est déjà fait.

## L'Éditeur « Nano »

**Nano est un éditeur de texte, pas un traitement de texte !**
Savez-vous vraiment ce qu'est un éditeur de texte ? Ne le confondez-vous pas avec un traitement de texte ? Un **éditeur de texte** est un programme qui permet de modifier des fichiers de texte brut, sans mise en forme (gras, italique, souligné…). Sous Windows, on dispose d'un éditeur de texte très basique : le Bloc-Notes. Sous Linux, on a le choix entre Nano, Vim, Emacs et bien d'autres, sachant qu'au moins un de ceux-là est installé par défaut sur la plupart des distributions.
Le symbole ^ signifie Ctrl (la touche Contrôle de votre clavier). Ainsi, pour quitter Nano, il suffit de taper Ctrl + X.
Voici les raccourcis les plus importants :

- Ctrl + G : afficher l'aide ;
- Ctrl + K : couper la ligne de texte (et la mettre dans le presse-papier) ;
- Ctrl + U : coller la ligne de texte que vous venez de couper ;
- Ctrl + C : afficher à quel endroit du fichier votre curseur est positionné (numéro de ligne…) ;
- Ctrl + W : rechercher dans le fichier ;
- Ctrl + O : enregistrer le fichier (écrire) ;
- Ctrl + X : quitter Nano.

Vous pouvez vous déplacer dans le fichier avec les flèches du clavier ainsi qu'avec les touches Page Up et Page Down pour avancer de page en page (les raccourcis Ctrl + Y et Ctrl + V fonctionnent aussi).
Si l'aide-mémoire vous encombre, vous pouvez gagner de la place en appuyant sur Échap puis sur X. Vous pouvez l'afficher de nouveau avec la même suite de touches.

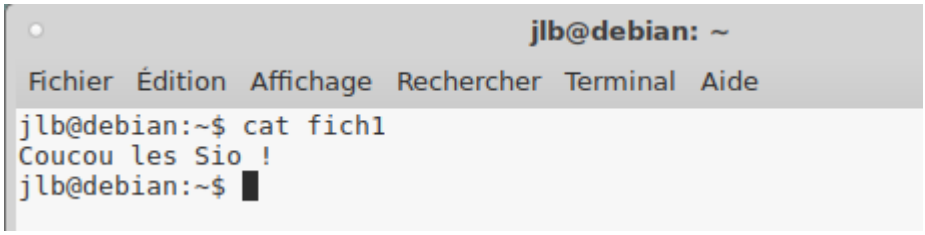
## La commande Cat

La commande **cat** constitue un éditeur (très) simplifié. Elle permet également d'afficher le contenu d'un fichier entier à l'écran.

**cat > fich1 (Enter)** **# Création d'un fichier fich1 avec le texte que vous tapez ensuite.**

Pour finir : (Enter)+ **CTRL+ D**

**cat fich1 (Enter)** **# Affiche le contenu (texte) d'un fichier**

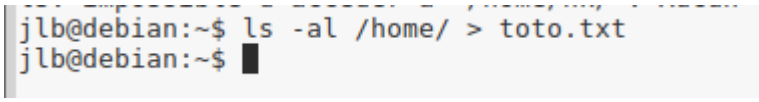



## Redirections

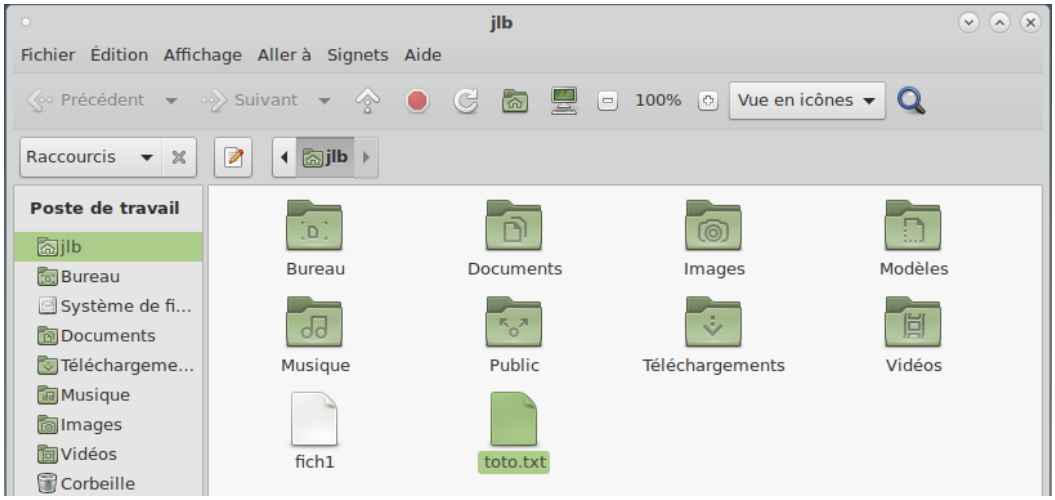
Linux, comme tout système de type Unix, possède des mécanismes permettant de rediriger les entrées-sorties standards vers des fichiers.

Ainsi, l'utilisation du caractère «>» permet de rediriger la sortie standard d'une commande située à gauche vers le fichier situé à droite :

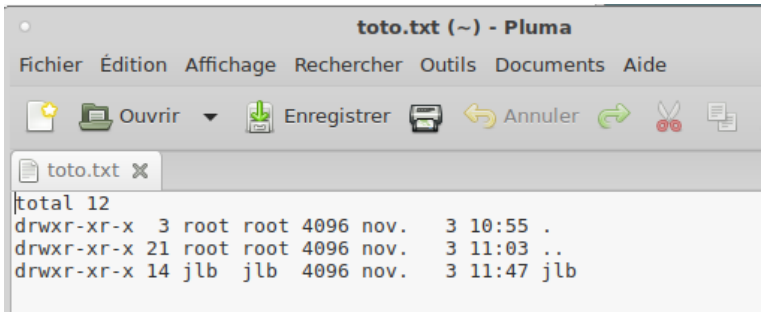
**# Création d'un fichier toto.txt contenant le résultat de la commande "ls –al"**
ls -al /home/ > toto.txt



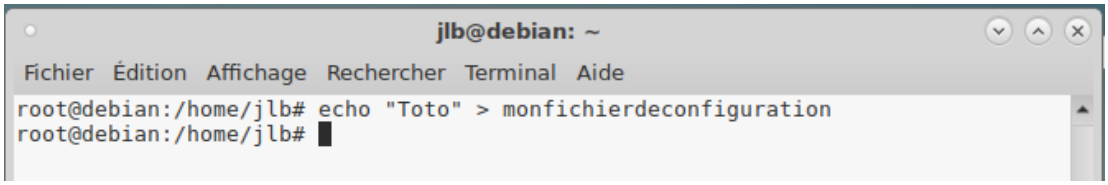

Vous trouverez le fichier toto.txt dans le Home




Cliquez dessus pour en lire le contenu :

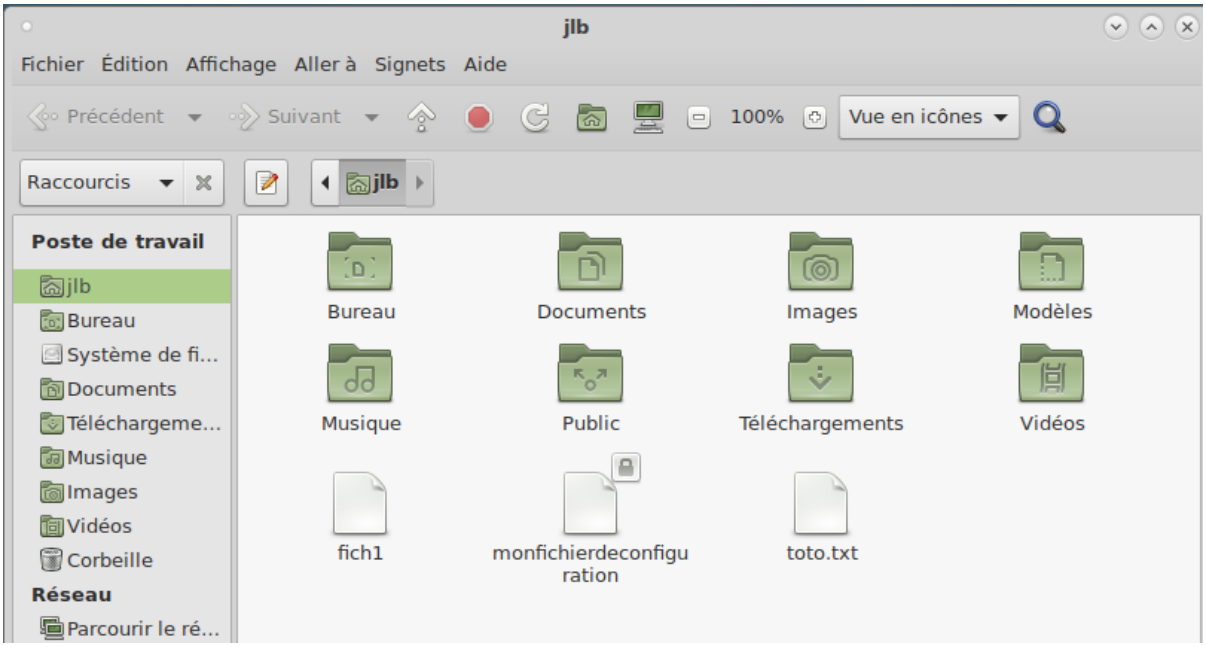


echo "Toto" > monfichierdeconfiguration

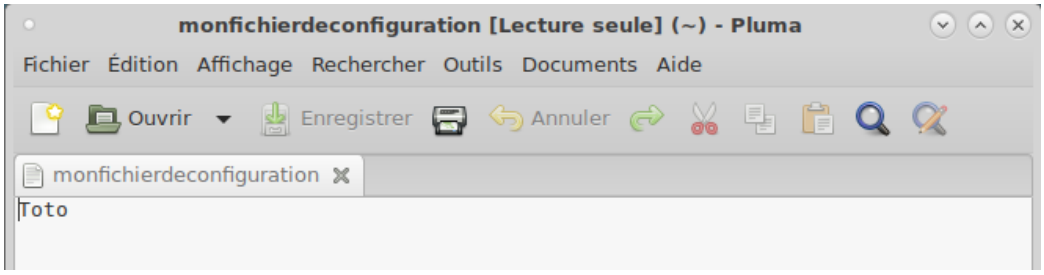



Ici, vous demandez à linux de créer un fichier nommé « monfichierdeconfiguration » contenant le texte « Toto »

Pour vérifier :




Cliquez sur « monfichierdeconfiguration » pour l'ouvrir




**<u>La commande echo est active en mode SU.</u>**

**Quelques utilisations pratiques**

- Pour écrire à la fin d'un fichier sans en écraser le contenu, on utilise les signes >> :

# echo "Ma jolie phrase est belle." >> monfichier

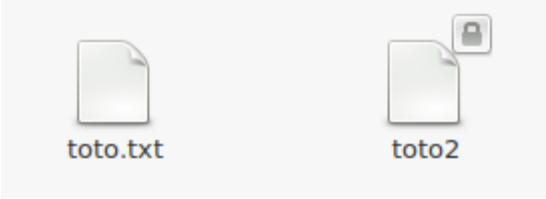
- Pour écraser un fichier, en effaçant tout son contenu, on utilise le signe > :

# echo "Ma jolie phrase est belle." > monfichier

La commande suivante est équivalente à une copie de fichiers :

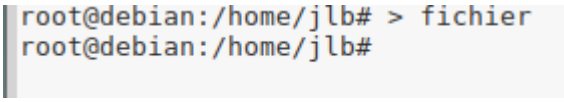
**cat toto > toto2**

Vous vérifiez la création du fichier de copie




La redirection «>» a pour but de créer un nouveau fichier. Ainsi, si un fichier du même nom existait, celui-ci sera écrasé. La commande suivante crée tout simplement un fichier vide :

**> fichier**




L'emploi d'un double caractère «>>» permet de concaténer la sortie standard vers le fichier, c'est-à-dire ajouter la sortie à la suite du fichier, sans l'écraser.

**cat toto.txt >> toto2**

De manière analogue, le caractère «<» indique une redirection de l'entrée standard. La commande suivante envoie le contenu du fichier toto.txt en entrée de la commande cat, dont le seul but est d'afficher le contenu sur la sortie standard (exemple inutile mais formateur) :

**cat < toto.txt**

## Liens physiques et liens symboliques

### Les liens physiques

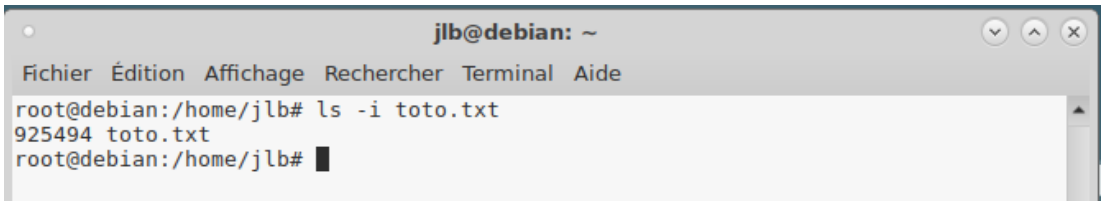
Le système identifie les fichiers (physiquement) par un identificateur unique qui s'appelle le numéro d'i-nœud (i-node).

L'i-nœud est en fait un numéro unique qui identifie le fichier :

Par exemple, pour le fichier **toto.txt** si vous faites :

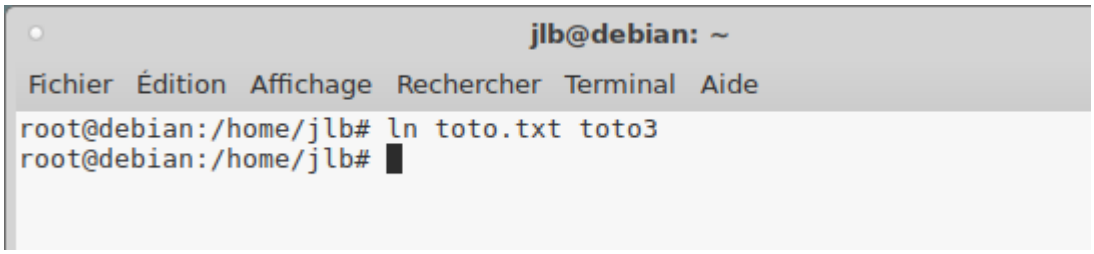
```
# ls -i toto.txt
```

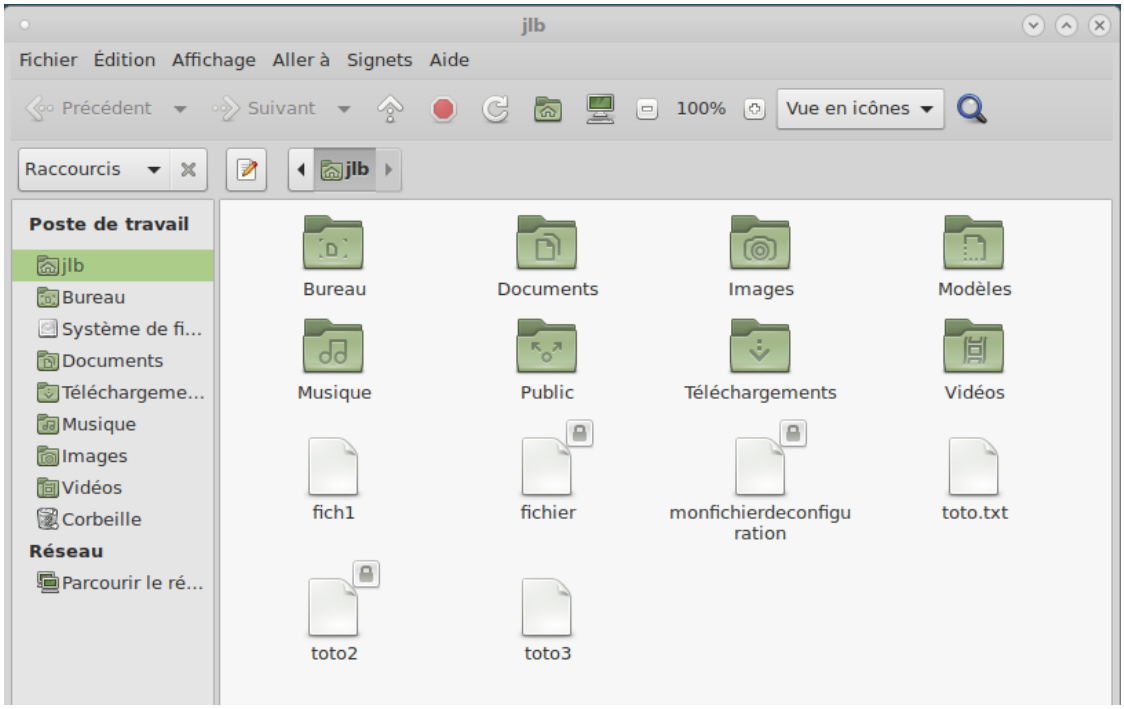
Vous obtiendrez son numéro i-node.




Pour créer un autre lien, on utilise la commande ln :

```
# ln toto.txt toto3
```



Comme vous le voyez un fichier toto3 vient d'apparaitre. Ouvrez-le, vous voyez qu il contient le texte de toto.txt. Ajouter du texte, enregistrez, ouvrez toto.txt. Que constatez-vous ?

Vérification :

```
# ls -i toto.txt toto3
```

Nous obtenons le même numéro

La commande ls -l indique le nombre de liens que comporte un fichier : C'est le chiffre venant après les permissions.

**Nota :** Un fichier n'existe plus dans le système de fichier lorsque son i-node arrive à 0. C'est un peu comme donner des « points de vie » à un fichier à chaque fois que vous créez un nouveau lien. Cela permet de simplifier l'accès à un fichier en créant des « images » de ce fichier dans plusieurs répertoires.

### Les liens symboliques

Cette sorte de lien permet de donner un autre nom au fichier, mais n'utilise pas l'i-nœud physique du fichier. C'est une sorte de raccourci et cela n'ajoute pas de « points de vie » au fichier.

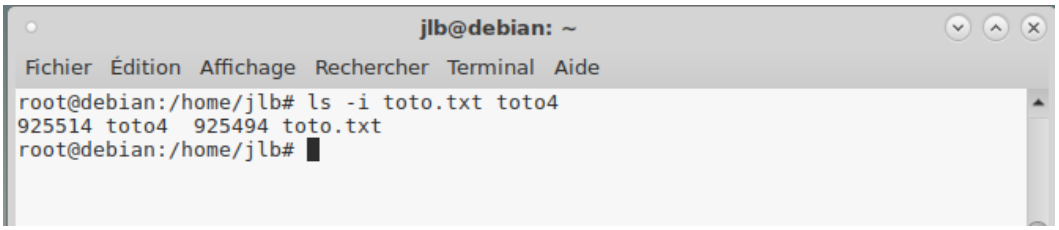
Pour créer un lien symbolique, il suffit de passer l'option **-s** à la commande **ln** :

```
# ln -s toto.txt toto4
```

Cela va créer un fichier /droopy/config avec un autre i-node :

Vérifiez avec :

```
# ls -i toto.txt toto4
```




Nous avons dans ce cas 2 numéros d'i-node différents.

```
# ls -l toto4
```

Indique que config pointe (-) bien sur ma.conf.

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# ls -l toto4
lrwxrwxrwx 1 root root 8 nov.   3 13:23 toto4 -> toto.txt
root@debian:/home/jlb#
```

Avec le lien symbolique, les permissions de toto4 seront les mêmes que pour toto.txt.

Il est donc facile d'identifier le lien symbolique d'un fichier avec la commande ls -l, alors qu'il n'en va pas de même pour un lien physique.

## Compression et archivage de fichiers : Gzip, Zip, et Tar

### Compression avec Gzip et Zip

Les fichiers comprimés utilisent moins d'espace disque et se téléchargent plus rapidement que les grands fichiers non comprimés. Vous pouvez comprimer les fichiers Linux à l'aide de l'instrument de compression open-source Gzip ou Zip, qui est reconnu par la plupart des systèmes d'exploitation.

Par convention, les fichiers comprimés se voient attribuer l'extension .gz. La commande Gzip crée un fichier comprimé terminant par .gz; Gunzip extrait les fichiers comprimés et efface le fichier .gz.

Pour comprimer un fichier, entrez la commande suivante à l'invite du shell : gzip filename.ext

Le fichier sera comprimé et sauvegardé comme filename.ext.gz.

Pour décompresser un fichier comprimé, tapez : gunzip filename.ext.gz

Le filename.ext.gz est effacé et remplacé par filename.ext.

Si vous échangez des fichiers avec des utilisateurs non LINUX, vous devriez utiliser zip pour éviter les problèmes de compatibilité. Linux peut facilement ouvrir des fichiers zip ou gzip, mais les systèmes d'exploitation non-Linux pourraient avoir des problèmes avec les gzip.

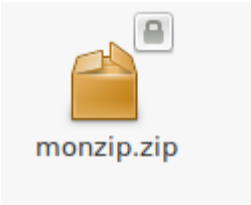
Pour comprimer un fichier à l'aide de zip, entrez ceci :

**zip -r filename.zip files**

Dans cet exemple, filename représente le fichier que vous créez, et files représente les fichiers que vous voulez placer dans le nouveau fichier :

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# zip -r monzip toto.txt
  adding: toto.txt (deflated 47%)
root@debian:/home/jlb#
```

Vous devez pouvoir voir un fichier nommé monzip.zip



Pour extraire le contenu d'un fichier zip, entrez : unzip filename.zip

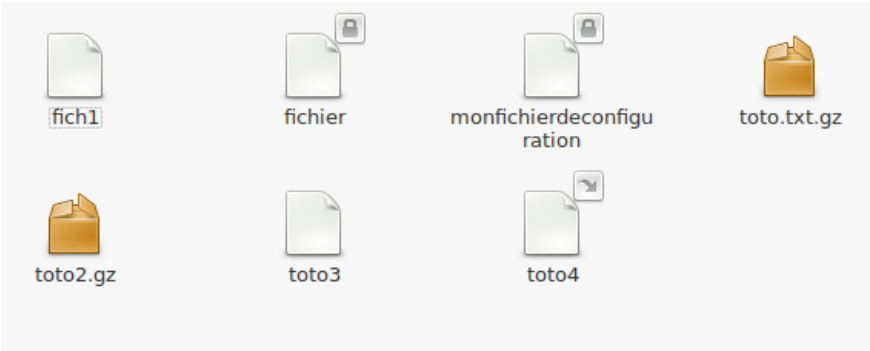
```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# zip -r monzip toto.txt
  adding: toto.txt (deflated 47%)
root@debian:/home/jlb# unzip monzip.zip
Archive:  monzip.zip
replace toto.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: toto.txt
root@debian:/home/jlb#
```

Vous pouvez compresser plusieurs fichiers en même temps avec zip ou gzip. Énumérez les fichiers en les séparant par un espace :

**gzip toto.txt toto2**

La commande ci-dessus compresse toto.txt, toto2

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# gzip toto.txt toto2
root@debian:/home/jlb#
```



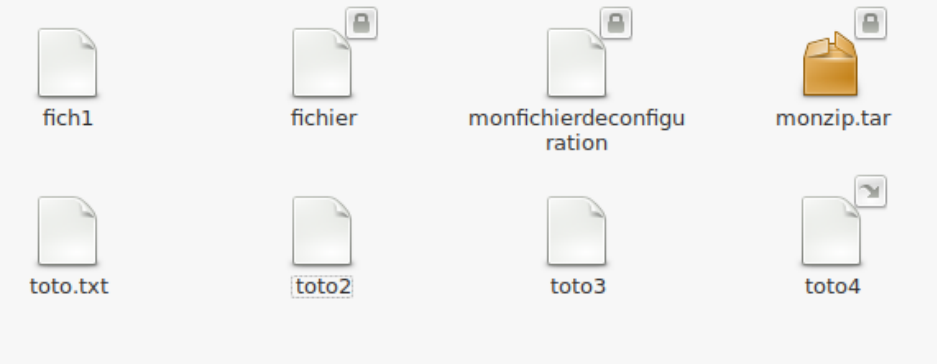
## Archiver avec Tar

Les fichiers **tar** placent plusieurs fichiers ou le contenu d'un répertoire ou de plusieurs répertoires dans un seul fichier. Il s'agit d'une bonne manière de créer des sauvegardes et des archives. Généralement, les fichiers tar terminent par l'extension .tar.

Pour créer un fichier tar, tapez :

**tar cvf monzip.tar toto.txt toto2**

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# tar cvf monzip.tar toto.txt toto2
toto.txt
toto2
root@debian:/home/jlb#
```




Dans cet exemple, monzip.tar représente le fichier que vous créez et toto.txt toto2 représente les fichiers que vous voulez placer dans le nouveau fichier.

Il est possible de compresser des dossiers dans ce cas la commande est de ce type :

**tar -cvf File.tar /dossier**

Pour afficher la liste du contenu d'un fichier tar, entrez :

**tar -tvf monzip.tar**

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# tar -tvf monzip.tar
-rw-r--r-- jlb/jlb         156 2017-11-03 13:54 toto.txt
-rw-r--r-- jlb/jlb         300 2017-11-03 13:54 toto2
root@debian:/home/jlb#
```

Pour extraire le contenu d'un fichier tar, entrez :

**tar -xvf monzip.tar**

Cette commande n'élimine pas le fichier .tar, mais elle place des copies du contenu de .tar dans le répertoire dans lequel vous travaillez actuellement.

La commande tar ne compresse pas automatiquement les fichiers. Vous pouvez compresser les fichiers tar avec :

**tar -czvf monzip.tar**

Les fichiers tar compressés se voient attribuer l'extension .tgz et sont comprimés avec gzip.

Pour décompresser un fichier tar, entrez :

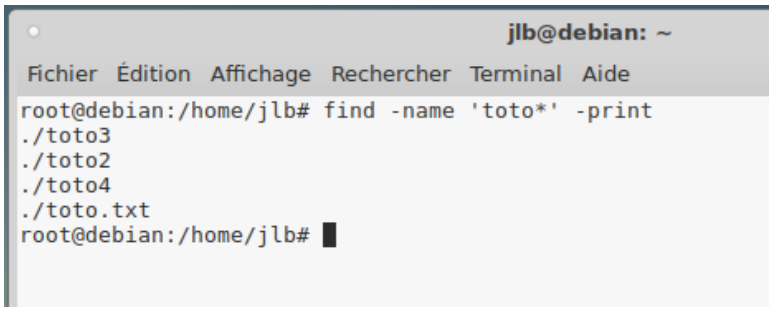
**tar -xzvf monzip.tgz**

## La commande find

La commande find permet de trouver des fichiers dans un répertoire.

Exemple simple : comment trouver un fichier portant un nom donné ? ***find directory -name targetfile –print***

**[user@myHome]$ find / -name toto -print**




Décomposition de la commande de l'exemple :

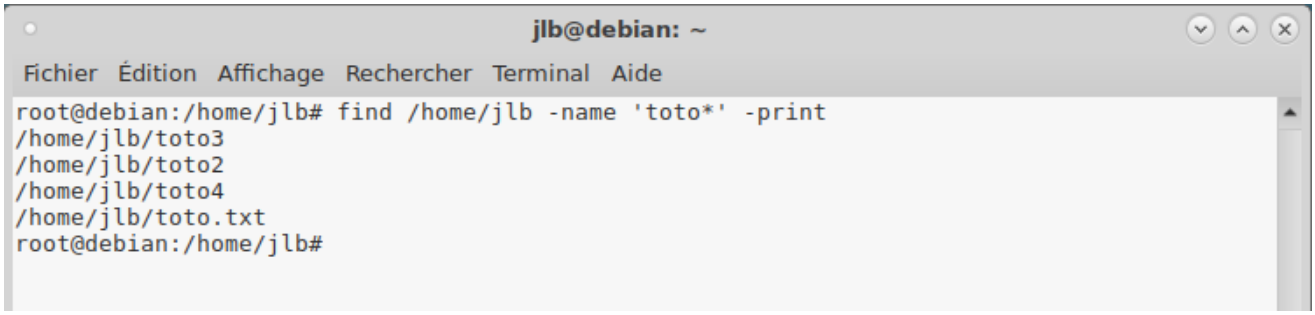
"/" indique que nous voulons chercher à partir de la racine notre fichier.

**-name** " est l'option qui indique ici que nous voulons spécifier le nom d'un fichier. **-print**" demande à find d'afficher le résultat.

Un peu long n'est-ce pas pour trouver la réponse dans tout cette grosse arborescence ? En général on recherche rarement un fichier depuis la racine.

Pour chercher tous les fichiers commencant par "toto" et définir à partir de quel répertoire on souhaite effectuer la recherche on utilise cette syntaxe :

**find /home/jlb -name 'toto*' -print**



# EXERCICES APPLICATIFS

*Remarque : Utiliser la commande man pour trouver l'aide sur la fonction ou la syntaxe d'une instruction.*

## Partie 1 : Gestion des fichiers et vi.

- Passez en mode console avec les touches Ctrl+Alt+F 1 (Alt + F7 pour revenir au mode graphique)

- Connectez-vous avec le compte root et son mot de passe.

- Placez-vous dans le répertoire /home et créez trois répertoires (rep1, rep2, rep3).

- Utiliser la commande ls pour voir le résultat.

- Placez-vous dans le répertoire rep1 et créez avec l'éditeur vi, un fichier texte nommé toto.

- Insérez du texte dans ce fichier et sauvegardez-le.

- Copiez ce fichier dans le deuxième répertoire /home/rep2 et dans le rep3 sous un autre nom (voir le résultat avec ls).

**Modifier le fichier toto du répertoire rep1 avec vi**

(Important : refaire plusieurs fois cette action pour bien maîtriser vi)

- Dans le fichier toto, utilisez les différentes commandes de vi pour chercher du texte, copier/coller un mot, une ligne, un paragraphe, supprimer une ligne, enregistrer les modifications. (Refaire plusieurs fois).

- Modifiez le fichier toto avec vi mais ne pas enregistrer les modifications, afficher le contenu de toto avec la commande cat

- Copiez ce fichier sous différents noms et supprimez-les (en restant dans le dossier rep1).

- Utilisez la commande pwd pour afficher le répertoire courant.

- Utilisez la commande find pour trouver les fichiers qui se nomme toto dans le répertoire /home.

- Utilisez la commande find pour trouver les fichiers dont le nom commence par t dans le répertoire /home.

- Utilisez la commande find pour trouver tous les fichiers dont le nom se termine par. conf dans le répertoire /etc en restant dans le répertoire rep1.

- Faire la même chose, mais retourner le résultat dans un fichier nommé listeconf (avec l'opérateur de redirection ~).

- Regardez le contenu de listeconf avec cat et avec vi.

- Utilisez la commande find pour trouver tous les fichiers dont le nom commence par host dans le répertoire /etc.

- Avec vi, visualiser le contenu du fichier nommé hosts situé dans /etc, que contient-il ?

- Placez-vous dans le répertoire /etc et utilisez la commande ls avec les différentes options -l, -a, |more (tube).

- Tapez la commande cd et utiliser la commande pwd pour trouver dans quel répertoire vous êtes.

- Placez-vous dans le répertoire rep1 et créez un lien physique du fichier toto, qui sera situé dans /home/rep3/toto2 avec la commande ln

- Modifiez le fichier toto avec vi

- Placez-vous dans rep3 et ouvrir et modifier le fichier toto2 avec vi, (que contient toto2 ?)

- Placez-vous dans rep1 et vérifier le contenu de toto (que contient toto ?)

- Supprimer le fichier toto, vérifié que toto2 dans rep3 existe toujours, copier toto2 dans rep1 avec le nom toto1

- Placez-vous dans rep3 et créez un lien symbolique du fichier /home/rep1/toto1, qui se nomme stoto1 dans rep3

- Utilisez la commande ls -l, comment apparaît stoto1 ?

- Utiliser la commande readlink toto1, quel est le résultat ? supprimer toto1.

# SYSTEMES DE GESTION DE FICHIERS (SUITE)

## Voir un fichier (cat et more)

La commande cat permet de lire des fichiers. Nous avons vu précédemment que le répertoire /root contenait des fichiers de configuration. Ces fichiers sont simplement des fichiers textes avec un agencement et une syntaxe particulière.

Regardons le contenu du fichier .bashrc qui permet de configurer à souhait son shell :

*]# cat .bashrc # .bashrc*

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# cat .bashrc # .bashrc
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples

# If not running interactively, don't do anything
case $- in
    *i*) ;;
      *) return;;
esac

# don't put duplicate lines or lines starting with space in the history.
# See bash(1) for more options
HISTCONTROL=ignoreboth

# append to the history file, don't overwrite it
shopt -s histappend

# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000

# check the window size after each command and, if necessary,
# update the values of LINES and COLUMNS.
```

Une option utile de cat est -n qui permet de numéroter les lignes (ne pas oublier que cat permet de lire et non de modifier un fichier. Ainsi la numérotation de ligne apparaît à l'écran mais le fichier .bashrc n'en est pas pour autant modifié).

*# cat -n .bashrc*

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# cat -n .bashrc
     1  # ~/.bashrc: executed by bash(1) for non-login shells.
     2  # see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
     3  # for examples
     4
     5  # If not running interactively, don't do anything
     6  case $- in
     7      *i*) ;;
     8        *) return;;
     9  esac
    10
    11  # don't put duplicate lines or lines starting with space in the history.
    12  # See bash(1) for more options
    13  HISTCONTROL=ignoreboth
    14
    15  # append to the history file, don't overwrite it
    16  shopt -s histappend
    17
```

Si vous souhaitez connaître les autres options de cat, tapez au prompt "cat --help".

Vous pouvez utiliser la commande **more** pour visualiser un fichier. La commande more a l'avantage d'afficher le fichier page par page. Pour passer d'une page à l'autre, tapez sur la touche ESPACE.

## Les commande head et tail

La commande **tail** est tout simplement inévitable.

Elle permet d'afficher les dernières lignes d'un fichier. Jusque-là on pourrait se dire qu'après tout il suffit d'éditer le fichier et de se déplacer à la fin. D'une part c'est une méthode fastidieuse mais d'autre part, l'option -f va définitivement vous convaincre de l'utiliser :

L'option -f demande à tail de ne pas s'arrêter lorsqu'elle a affiché les dernières lignes du fichier et de continuer à afficher la suite du fichier au fur et à mesure que celui-ci grossit jusqu'à ce que l'utilisateur interrompe la commande avec la combinaison de touches d'interruption Ctrl+c.

La commande **head** réalise la même chose que tail mais elle affiche les premières lignes du fichier au lieu d'afficher les dernières. tail et head ont une option commune qui permet d'afficher le nombre de ligne que l'on souhaite :

**tail -5 nom_du_fichier" affichera les 5 dernières lignes du fichier**
**head -15 nom_du_fichier" affichera les 15 premières lignes du fichier.**

**Par défaut, tail et head affichent 10 lignes.**

## Exercice :

Dans le répertoire d'accueil, créez un répertoire **rep**.

Dans ce sous répertoire, créez trois fichiers toto.c, tata.h et lala.o.

À partir du répertoire d'accueil, en utilisant la commande **find** affichez tous les fichiers du répertoire **rep** commencent par la lettre t.

Quelle est la commande à utiliser ? **find . –name 't*' –print** *

De même, toujours à partir du répertoire d'accueil, en utilisant la commande find, affichez tous les fichiers du répertoire **rep** de type .c

Quelle est la commande à utiliser ? **find rep/ –name '*.c' –print** *

De même, toujours à partir du répertoire d'accueil, en utilisant la commande find, affichez à l'écran le contenu de tous les fichiers du répertoire **rep** de type .c

Quelle est la commande à utiliser ? **find . –name '*.c' –exec cat '{}'** \;

**Nota :** Vous pouvez stopper le défilement avec C-c !

## La commande locate

La commande locate a la même mission que find. Pourtant vous verrez qu'en utilisant la commande locate, le fichier sera trouvé beaucoup plus rapidement. Pourquoi ? Parce que locate ne va pas chercher le fichier dans toute l'arborescence des répertoires mais va localiser la position du fichier dans une base de données qui contient la liste des fichiers existants. Cette base de données est en général automatiquement générée une fois par jour par le système grâce à une commande appelée updatedb. Sur un système Linux, cette base de données se trouve dans le répertoire /usr/lib et se nomme locatedb.

La syntaxe est donc simple: [user@myHome]$ locate nom_du_fichier

Bien que la commande locate soit très intéressante, elle ne possède pas la puissance des options de find. De plus, si vous créez des fichiers pendant la journée et que vous les recherchez avec la commande locate, il n'est pas sûr que la base de données ait été remise à jour. Bref, locate est un complément de find.

## La commande which

which vous permet simplement de connaître le chemin d'un exécutable.

Exemple :

**[user@myHome]$ which ls (question)**
**[user@myHome]$ /bin/ls (réponse)**

## La commande file

La commande file permet d'afficher le type du fichier (exécutable, répertoire, ASCII, ...)

**file /bin/ls (question)**
**file /etc/passwd file /usr (question)**

## Trouver du texte dans un fichier : commande grep

La commande grep est un pivot des commandes UNIX. Elle cherche une expression rationnelle dans un ou plusieurs fichiers, exemple :

**$ grep test toto.txt**

```
root@debian:/home/jlb# grep test toto.txt
test
root@debian:/home/jlb#
```

grep est la commande qui vous fouille les fichiers

La commande a donc affiché la ligne qui contient le mot "fouille" dans le fichier linux-commande.html.

La richesse de la commande grep permet de faire des recherches sur plusieurs fichiers et d'avoir un format de sortie adéquat.

Par exemple, il serait agréable de savoir où se trouve cette ligne qui contient le mot fouille dans le fichier :

**$ grep -n test toto.txt**

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# grep -n test toto.txt
6:test
root@debian:/home/jlb#
```

Le mot fouille se trouve à la ligne numéro 6 et c'est l'option -n qui nous a permis de connaître ce numéro.

Une autre option très utile est -rn qui permet d'afficher les noms des fichiers contenant ce que l'on cherche :

**$ grep -rn test**

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# grep -rn test
.bashrc:77:    test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "$(dircolors -b)"
toto3:6:test
toto.txt:6:test
root@debian:/home/jlb#
```

Ici, j'ai demandé à la commande grep de chercher l'occurrence "test" dans les fichiers du répertoire courant.

Le résultat est le nom des fichiers qui contiennent l'occurrence.

**EXERCICE :**

Copiez le fichier stdio.h situé dans /usr/include dans votre répertoire courant.

Quelle commande faut-il utiliser ?

À l'aide de la commande **grep,** affichez les lignes du fichier stdio.h qui contiennent la chaîne ***define***.

Quelle commande utiliser ? **grep 'define' studio.h**

Utilisez la commande grep pour isoler la ligne dans le fichier /etc/passwd qui contient vos détails.

Quelle est la commande à utiliser et quel est le résultat obtenu ? **grep 'login' /etc/passwd**

En utilisant la commande find ainsi que la commande grep, affichez la liste de tous les fichiers du répertoire courant et de ses sous répertoires qui ont une extension .txt.

Quelle est la commande à utiliser ? **find . | frep '*.txt'** *

## La commande diff

diff nous permet de découvrir les différences entre deux versions d'un fichier. Pour situer les différences dans leurs contextes, on vous conseille d'utiliser l'option -c.

***diff [-c] fichier fichier***

**$ diff -c toto.txt toto2**

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# diff -c toto.txt toto2
*** toto.txt     2017-11-03 14:27:58.037434180 +0100
--- toto2        2017-11-03 13:54:37.360491052 +0100
***************
*** 2,7 ****
  drwxr-xr-x  3 root root 4096 nov.   3 10:55 .
  drwxr-xr-x 21 root root 4096 nov.   3 11:03 ..
  drwxr-xr-x 14 jlb  jlb  4096 nov.   3 11:47 jlb
!
! test
!
--- 2,8 ----
  drwxr-xr-x  3 root root 4096 nov.   3 10:55 .
  drwxr-xr-x 21 root root 4096 nov.   3 11:03 ..
  drwxr-xr-x 14 jlb  jlb  4096 nov.   3 11:47 jlb
! total 12
! drwxr-xr-x  3 root root 4096 nov.   3 10:55 .
! drwxr-xr-x 21 root root 4096 nov.   3 11:03 ..
! drwxr-xr-x 14 jlb  jlb  4096 nov.   3 11:47 jlb
root@debian:/home/jlb#
```

Les lignes marquées par un point d'exclamation ! sont les lignes qui ne sont pas identiques.

Parce diff risque d'afficher beaucoup plus de lignes que l'écran (ou fenêtre) puisse accommoder, on fait souvent un tube (pipe) qui dirige les résultats de diff à la commande more. Par exemple :

**$ diff -c toto.txt toto2 | more**

## Redirection entrée/sorties et pipes

**La connexion de plusieurs commandes : les pipes**

Qu'est-ce qu'un "pipe" (parfois appelé « tube ») ? Si on le décrit ce n'est rien d'autre que cette barre verticale que vous pouvez obtenir avec la combinaison de touches "Altgr + 6" sur les claviers français classiques. Un tube permet de passer le résultat d'une commande à autre commande. Un exemple permettra de comprendre tout cela beaucoup plus facilement

Je veux savoir quels sont tous les processus "bash" qui fonctionnent sur le système, mais je veux que la commande ps aux ne me fournisse que les lignes qui contiennent le mot "bash" pour m'éviter d'avoir à parcourir toute la longue liste qu'affiche ps aux :

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# ps aux | grep bash
jlb       9341  0.0  0.1   5676  3520 pts/0    Ss   14:16   0:00 bash
root      9345  0.0  0.1   5672  3700 pts/0    S    14:16   0:00 bash
root      9441  0.0  0.0   4728   832 pts/0    S+   14:40   0:00 grep bash
root@debian:/home/jlb#
```

On peut dire que l'on a "connecté" deux commandes entre elles. Mais vous pouvez ainsi en connecter autant que vous voulez en utilisant cette syntaxe :

commande1 | commande2 | commande3 ... | commandeN Si on se rend compte de l'utilité des pipes, progressivement on les utilise et on finit par ne plus s'en passer.

## Connaître l'espace disque utilisé (df et du)

La commande df permet de connaître l'emplacement de montage des systèmes de fichiers (partitions utilisables pour stocker des fichiers) accessibles sur votre système et les capacités restantes sur chacun d'eux.

$ DF

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# df
Sys. de fichiers blocs de 1K Utilisé Disponible Uti% Monté sur
udev                  1554432       0    1554432   0% /dev
tmpfs                  312664    5080     307584   2% /run
/dev/sda1            27706468 3432708   22843284  14% /
tmpfs                 1563304       0    1563304   0% /dev/shm
tmpfs                    5120       4       5116   1% /run/lock
tmpfs                 1563304       0    1563304   0% /sys/fs/cgroup
tmpfs                  312660      32     312628   1% /run/user/1000
root@debian:/home/jlb#
```

La commande **du** permet de connaître l'utilisation disque en kilo-octet par le répertoire spécifié et ses sous répertoires.

**$ du ./config**

```
root@debian:/home# du ./jlb
4       ./jlb/.cache/mate/background
8       ./jlb/.cache/mate
12      ./jlb/.cache
12      ./jlb/.local/share/keyrings
76      ./jlb/.local/share/gvfs-metadata
4       ./jlb/.local/share/Trash/files
4       ./jlb/.local/share/Trash/expunged
4       ./jlb/.local/share/Trash/info
16      ./jlb/.local/share/Trash
112     ./jlb/.local/share
116     ./jlb/.local
4       ./jlb/Documents
60      ./jlb/.config/pulse
4       ./jlb/.config/enchant
4       ./jlb/.config/caja/scripts
24      ./jlb/.config/caja
20      ./jlb/.config/pluma
12      ./jlb/.config/dconf
132     ./jlb/.config
4       ./jlb/Bureau
4       ./jlb/Public
4       ./jlb/Musique
4       ./jlb/Images
4       ./jlb/.gnupg/private-keys-v1.d
8       ./jlb/.gnupg
4       ./jlb/Modèles
4       ./jlb/Vidéos
4       ./jlb/Téléchargements
404     ./jlb
root@debian:/home#
```

# CONTROLE DES PROCESSUS

Il se peut qu'une commande ne termine pas ou dure trop longtemps.

Il y aussi des commandes comme xterm qui ne se terminera que si l'on termine son propre shell (par exit).

Enfin certaines commandes sont volontairement conçues pour ne pas terminer, comme yes ou xeyes, par exemple.

Normalement, l'interpréteur de commandes attend la fin de la commande en cours avant d'en accepter une nouvelle et il se trouve donc bloqué dans les situations envisagées ci-dessus. Heureusement, on dispose de plusieurs moyens d'action sur la commande en cours.

Le plus brutal, est Ctrl+c (Control c) qui interrompt définitivement la commande, sans possibilité de reprise.

On peut aussi suspendre la commande courante par Ctrl+z.

Une commande que l'on vient de suspendre peut être relancée par la commande **fg** (comme foreground).

On peut également la relancer en arrière-plan par la commande **bg** (comme background). Une commande qui s'exécute en arrière-plan laisse l'interpréteur de commande disponible.

Une commande peut être directement lancée en arrière-plan en ajoutant un & à la fin. Par exemple, la commande :

xterm &

est équivalente à la séquence suivante :

xterm

Ctrl+z

bg

Et si on a lancé une commande comme sleep 1000 &, que peut-on faire ? Il est clair que C-c (contrôle+C) ou C-z n'agissent que sur la commande de premier-plan.

Et, comme on peut avoir plusieurs commandes en arrière-plan, il faut pouvoir désigner celle sur laquelle on veut agir. On lance alors la commande ps qui nous donne le pid du processus concerné que l'on utilise ensuite dans une commande kill appropriée, voir ps et kill ci-dessous.

On remarquera que l'on peut ainsi tuer son tcsh ... mais on s'en lasse vite ! Les commandes indispensables :

C-c

Interrompt définitivement la commande de premier plan, sans possibilité de reprise.

C-z

Suspend la commande de premier plan. Elle peut être relancée par les commandes fg ou bg.

fg

La commande fg permet de relancer au premier plan la dernière commande, qu'elle soit suspendue ou non (cas d'une commande d'arrière-plan).

bg

La commande bg permet de relancer en arrière-plan la dernière commande suspendue.

ps

La commande ps, utilisée sans argument, liste tous les processus attachéscl au terminal. Chaque ligne affichée commence par le PID qui est le numéro qui identifie chaque processus dans le système. On repère ainsi le pid d'un processus que l'on pourra supprimer par la commande kill.

kill

La commande kill s'utilise en général avec une option qui indique l'action voulue et prend en paramètre le pid du processus concerné. On utilise surtout :

kill -KILL pid qui interrompt définitivement

ou

kill -STOP pid qui suspend, on peut relancer ensuite par kill -CONT pid.

Une commande instructive

La commande top permet d'observer la vie d'un système Unix et de détecter les processus "gourmands". L'essai est particulièrement instructif sur la machine poly mais il ne faut pas en abuser car c'est une commande qui charge fortement le système.

## La commande "top" :

La commande top vous permet d'afficher des informations en continu sur l'activité du système. Elle permet surtout de suivre les ressources que les processus utilisent (quantité de RAM, pourcentage de CPU, la durée de ce processus depuis son démarrage).

Vous pourrez utiliser l'option -d pour spécifier des délais de rafraîchissement (en secondes).

En cours d'utilisation de top, il est possible de stopper un process de manière interactive en tapant k. top demande ensuite quel signal il doit envoyer : 15 (SIGTERM) est le signal par défaut qui met fin à un process, 9 (SIGKILL) est plus brutal.

Pour quitter top, normalement, appuyer simplement sur la touche "q". Sinon, pour essayer, vous pouvez faire un « kill » du process TOP…

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
top - 14:45:37 up  3:46,  1 user,  load average: 0,00, 0,00, 0,00
Tasks: 128 total,   1 running, 127 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0,2 us,  0,2 sy,  0,0 ni, 99,7 id,  0,0 wa,  0,0 hi,  0,0 si,  0,0 st
KiB Mem :  3126608 total,  2259252 free,   224148 used,   643208 buff/cache
KiB Swap:  3172348 total,  3172348 free,        0 used.  2581380 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
    1 root      20   0   27020   6332   5184 S   0,0  0,2   0:01.08 systemd
    2 root      20   0       0      0      0 S   0,0  0,0   0:00.00 kthreadd
    3 root      20   0       0      0      0 S   0,0  0,0   0:00.59 ksoftirqd/0
    5 root       0 -20       0      0      0 S   0,0  0,0   0:00.00 kworker/0:0H
    6 root      20   0       0      0      0 S   0,0  0,0   0:00.44 kworker/u4:0
    7 root      20   0       0      0      0 S   0,0  0,0   0:01.02 rcu_sched
    8 root      20   0       0      0      0 S   0,0  0,0   0:00.00 rcu_bh
    9 root      rt   0       0      0      0 S   0,0  0,0   0:00.02 migration/0
   10 root       0 -20       0      0      0 S   0,0  0,0   0:00.00 lru-add-drain
   11 root      rt   0       0      0      0 S   0,0  0,0   0:00.04 watchdog/0
   12 root      20   0       0      0      0 S   0,0  0,0   0:00.00 cpuhp/0
   13 root      20   0       0      0      0 S   0,0  0,0   0:00.00 cpuhp/1
   14 root      rt   0       0      0      0 S   0,0  0,0   0:00.01 watchdog/1
   15 root      rt   0       0      0      0 S   0,0  0,0   0:00.04 migration/1
   16 root      20   0       0      0      0 S   0,0  0,0   0:00.34 ksoftirqd/1
   18 root       0 -20       0      0      0 S   0,0  0,0   0:00.00 kworker/1:0H
   19 root      20   0       0      0      0 S   0,0  0,0   0:00.00 kdevtmpfs
   20 root       0 -20       0      0      0 S   0,0  0,0   0:00.00 netns
   21 root      20   0       0      0      0 S   0,0  0,0   0:00.00 khungtaskd
   22 root      20   0       0      0      0 S   0,0  0,0   0:00.00 oom_reaper
   23 root       0 -20       0      0      0 S   0,0  0,0   0:00.00 writeback
   24 root      20   0       0      0      0 S   0,0  0,0   0:00.00 kcompactd0
   25 root      25   5       0      0      0 S   0,0  0,0   0:00.00 ksmd
   27 root      39  19       0      0      0 S   0,0  0,0   0:00.00 khugepaged
   28 root       0 -20       0      0      0 S   0,0  0,0   0:00.00 crypto
   29 root       0 -20       0      0      0 S   0,0  0,0   0:00.00 kintegrityd
   30 root       0 -20       0      0      0 S   0,0  0,0   0:00.00 bioset
```

## La commande "ps" :

La commande ps permet de connaître les processus actifs à un moment donné :

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home# ps
  PID TTY          TIME CMD
 9344 pts/0    00:00:00 su
 9345 pts/0    00:00:00 bash
 9456 pts/0    00:00:00 ps
root@debian:/home#
```

Le "PID" est l'identificateur d'un processus, c'est un nombre. Chaque processus est identifié dans le système par un nombre unique.

Le "TTY" indique à quel port de terminal est associé le processus.

"STAT" indique l'état dans lequel se trouve le processus. Dans l'exemple, trois processus sont endormis (S comme "sleep"), et un processus en cours d'exécution (R comme "run"). Le processus qui est en cours d'exécution n'est autre que la commande "ps" que nous venons de lancer.

Le "TIME" indique depuis combien de temps le processus utilise les ressources du microprocesseur.

Le "COMMAND" précise, comme son nom l'indique, la commande dont l'état est décrit par PID, TTY, STAT et TIME.

Ceci dit, une simple commande "ps" n'indique pas tous les processus du système. Le simple fait de lancer ps nous a juste indiqué les processus associés à un terminal et qui dépendent de l'utilisateur courant (ici " user ").

En fait, il est tout à fait probable que d'autres processus non liés à un terminal aient été lancés par " user ".

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home# ps -x
  PID TTY      STAT   TIME COMMAND
    1 ?        Ss     0:01 /sbin/init
    2 ?        S      0:00 [kthreadd]
    3 ?        S      0:00 [ksoftirqd/0]
    5 ?        S<     0:00 [kworker/0:0H]
    6 ?        S      0:00 [kworker/u4:0]
    7 ?        S      0:01 [rcu_sched]
    8 ?        S      0:00 [rcu_bh]
    9 ?        S      0:00 [migration/0]
   10 ?        S<     0:00 [lru-add-drain]
   11 ?        S      0:00 [watchdog/0]
   12 ?        S      0:00 [cpuhp/0]
   13 ?        S      0:00 [cpuhp/1]
   14 ?        S      0:00 [watchdog/1]
   15 ?        S      0:00 [migration/1]
   16 ?        S      0:00 [ksoftirqd/1]
   18 ?        S<     0:00 [kworker/1:0H]
   19 ?        S      0:00 [kdevtmpfs]
   20 ?        S<     0:00 [netns]
   21 ?        S      0:00 [khungtaskd]
   22 ?        S      0:00 [oom_reaper]
   23 ?        S<     0:00 [writeback]
   24 ?        S      0:00 [kcompactd0]
```

Les commandes qui ne sont pas associées à un terminal sont reconnaissables par le point d'interrogation qui remplit le champ TTY.

Si vous voulez connaître tous les processus de la machine de tous les utilisateurs, il suffit d'utiliser l'option ax. Si en plus vous voulez connaître les utilisateurs associés à chaque processus, il vous suffit d'utiliser l'option aux. Vous verrez alors plusieurs colonnes s'ajouter dont "USER" qui indique à quel utilisateur appartient le processus. "%CPU" indique en pourcentage les ressources du microprocesseur utilisées par le processus. "%MEM" montre en pourcentage les ressources en mémoire vive utilisées par le processus. "RSS" donne réellement la mémoire utilisée en kilobytes par le processus. "START" indique l'heure à laquelle le processus a été lancé.

## La commande "kill" :

La commande "kill" permet d'expédier un signal à un processus en cours. Sa syntaxe est la suivante :

10

kill [options] PID

Par exemple, si j'ai lancé une connexion à l'Internet en PPP, un processus pppd sera en cours. Pour tuer le processus, je peux d'abord faire un ps -ax pour connaître le numero du PID de pppd et ensuite si par exemple le PID est 592, je peux tuer la connexion en faisant :

**]# kill 592**

Vous remarquerez que je suis logué en utilisateur "root" pour faire ceci, en effet le processus pppd appartenait à l'utilisateur "root" et un autre utilisateur ne peut pas lui expédier de signal.

Si un processus vous résiste, c'est à dire que vous n'arrivez pas à le tuer, vous devez utiliser la commande : kill -9 PID (PID étant toujours le numéro de de processus).

La commande "killall" permet aussi de tuer un processus mais au lieu d'indiquer le PID vous indiquerez le nom du processus.

Mais attention, plusieurs processus peuvent utiliser la même commande. Ainsi, si vous tapez :

**# killall grep**

Vous tuerez tous les processus qui contiennent la commande grep. Je vous recommande donc d'utiliser l'option "-i" qui vous demande une confirmation avant de tenter d'arrêter un processus.

# GESTION DES UTILISATEURS ET GROUPES

## LES UTILISATEURS

Linux est un système multi-utilisateur, plusieurs utilisateurs peuvent travailler sur le système de manière simultanée. Un utilisateur n'est pas obligatoirement rattaché à un individu physique. On parle de pseudo-utilisateur.

Un utilisateur possède un nom d'utilisateur appelé aussi login lui permettant de se connecter au SE. Ce login est rattaché à un mot de passe. Pour accéder aux ressources du SE, le couple login/mot de passe doit être correct, dans ce cas, nous utilisons le terme d'AUTHENTIFICATION.

Les utilisateurs sont identifiés par le SE grâce à un numéro unique nommé UID (identifiant utilisateur). Cela explique que si vous créez un compte toto et que vous donnez des droits sur certains fichiers, toto pourra y accéder. Maintenant supprimer le compte toto et recréez-le. Si vous essayez d'accéder aux fichiers précédents, cela ne fonctionnera pas car le toto précédent est différent du nouveau toto ? Pourquoi

Leur UID est différent et seule cette valeur compte pour le SE.

## LE FICHIER /ETC/PASSWD

Le fichier /etc/passwd contient des informations relatives à tous les utilisateurs du SE. La syntaxe de ce fichier est basique : une ligne contient les informations d'un utilisateur. Les champs composants une ligne sont :

**Login :** identifiant que l'utilisateur doit saisir pour s'authentifier. En général, pour des possibilités d'homonymie, la méthode de nommage la plus utilisé par les administrateurs est : première lettre du prénom suivie du nom complet avec l'ensemble en minuscule (pas d'importance sous Windows).

**Mot de passe :** ce mot de passe n'apparait pas en clair dans le fichier, il est chiffré en MD5 (méthode de cryptage). C'est la commande **passwd** qui chiffre le mot de passe. Les valeurs possibles sont : « * » impossibilité de se loguer avec ce compte, « !! » désactivation du compte, « x » ou « ! » le mot de passe se trouve dans un fichier protégé nommé shadow (voir plus loin). Si le champ est vide, cela indique que le compte n'a pas de mot de passe.

**UID :** Identifiant unique de l'utilisateur, root possède un UID 0, par convention, les UID <100 sont réservés aux comptes système.

**GID :** Identifiant du groupe principal de l'utilisateur, par défaut un utilisateur appartient à un groupe portant le même nom que lui.

**Informations utilisateurs :** Chaque information est séparée par un ; Très souvent apparait le nom et prénom complets de la personne mais l'administrateur est libre de stocker les informations utiles (en respectant les règles de la CNIL).

**Répertoire personnel :** Répertoire lié à l'utilisateur dans lequel sont stockés ses documents. Le chemin standard est /home/login.

**Shell :** interpréteur de commande lancé après l'authentification.

Voici la structure type d'une ligne dans ce fichier.

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
  GNU nano 2.7.4                Fichier : /etc/passwd

systemd-resolve:x:102:104:systemd Resolver,,,:/run/systemd/resolve:/bin/false
systemd-bus-proxy:x:103:105:systemd Bus Proxy,,,:/run/systemd:/bin/false
_apt:x:104:65534::/nonexistent:/bin/false
rtkit:x:105:109:RealtimeKit,,,:/proc:/bin/false
dnsmasq:x:106:65534:dnsmasq,,,:/var/lib/misc:/bin/false
messagebus:x:107:110::/var/run/dbus:/bin/false
usbmux:x:108:46:usbmux daemon,,,:/var/lib/usbmux:/bin/false
speech-dispatcher:x:109:29:Speech Dispatcher,,,:/var/run/speech-dispatcher:/bin/$
lightdm:x:110:111:Light Display Manager:/var/lib/lightdm:/bin/false
pulse:x:111:112:PulseAudio daemon,,,:/var/run/pulse:/bin/false
avahi:x:112:114:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/bin/false
jlb:x:1000:1000:jlb,,,:/home/jlb:/bin/bash
```

*Fig. 1 : Extrait du fichier passwd sous etc/passwd*

## LES GROUPES D'UTILISATEURS

Le fichier /etc/group est utilisé pour lister les groupes d'utilisateurs présents sur le S.E. Comme pour le fichier **/etc/passwd,** chaque ligne correspond à une occurrence (ici un groupe).

Chaque ligne peut contenir différentes informations concernant les groupes :

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
  GNU nano 2.7.4                Fichier : /etc/group

root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:
tty:x:5:
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
uucp:x:10:
man:x:12:
proxy:x:13:
kmem:x:15:
dialout:x:20:
fax:x:21:
voice:x:22:
cdrom:x:24:jlb
floppy:x:25:jlb
                          [ Lecture de 54 lignes ]
^G Aide      ^O Écrire    ^W Chercher  ^K Couper    ^J Justifier ^C Pos. cur.
^X Quitter   ^R Lire fich.^\ Remplacer ^U Coller    ^T Orthograp.^_ Aller lig.
```

*Fig. 4 : Extrait du fichier /etc/group*

**Groupe :** Le nom enregistré du groupe.

*** :** Lié au anciennes versions Unix, ce champ n'est plus utilisé sur les nouvelles versions, sous Ubuntu un x apparait.

**GID :** Identifiant unique du groupe (valeur numérique).

Dans l'extrait ci-dessus, nous pouvons constater que des groupes (Jean-Luc par exemple) ont été créés en même temps que les utilisateurs du même nom.

## Création d'un groupe

La création d'un groupe est possible grâce à la commande groupadd.

### DESCRIPTION

La commande groupadd crée un nouveau compte de groupe en utilisant les valeurs spécifiées sur la ligne de commande et les valeurs par défaut du système. Le nouveau groupe sera inséré dans les fichiers du système selon les besoins.

### OPTIONS

Les options disponibles pour la commande groupadd sont :

**-f**

Avec cette option, groupadd quittera juste avec un état de succès si le groupe indiqué existe déjà. Avec l'option -g, si le groupe indiqué existe déjà, un autre gid (non utilisé) sera choisi (c.-à-d. que -g est désactivée).

**-g GID**

La valeur numérique de l'identifiant du groupe (« group ID » ou GID). Cette valeur doit être unique, à moins que l'option -o ne soit utilisée. La valeur ne doit pas être négative. Par défaut le plus petit identifiant plus grand que 999 et que les identifiants des groupes existants est utilisé. Les valeurs comprises entre 0 et 999 sont généralement réservées aux comptes système.

**-h, --help**

Afficher un message d'aide et quitter.

**-K CLÉ=VALEUR**

Surcharger les valeurs par défaut du fichier /etc/login.defs (GID_MIN, GID_MAX et autres). L'option -K peut être indiquée plusieurs fois.

Exemple : -K GID_MIN=10 -K GID_MAX=499

**-o**

Permet de créer un groupe avec un identifiant (« GID ») déjà utilisé.

**AVERTISSEMENTS**

Les noms de groupe doivent commencer par une lettre minuscule ou un tiret bas (« underscore »), et seuls des lettres minuscules, des chiffres, des « underscore », des tirets ou un dollar peuvent suivre.

Soit, sous la forme d'une expression rationnelle : [a-z_][a-z0-9_-]*[$]

Les noms de groupe sont limités à 16 caractères.

## AJOUT D'UN UTILISATEUR

La commande **useradd** permet d'ajouter un utilisateur à la liste.

Certaines options utiles :

| | |
|---|---|
| -c | Insérer un commentaire sur cet utilisateur |
| -d | Indiquer le chemin du répertoire personnel de l'utilisateur (par defaut /home/login |
| -e | Indiquer une date d'expiration du compte. Le format est AAAA-MM-JJ |
| -g *groupeprincipal* | Indiquer le groupe principal d'un utilisateur. Le numéro par défaut est 1 |
| -G *groupe(s)supplémentaire(s)* | Indiquer le(s) groupe(s) principal d'un utilisateur. Si plusieurs groupes, il faut les séparer par des , |
| -s *cheminverslexe* | Indiquer un chemin vers un exécutable à lancer lors de la connexion de l'utilisateur |
| -u *UID* | Indiquer la valeur de l'UID |
| | |

## Création/suppression d'utilisateurs et de groupes

| | | |
|---|---|---|
| **groupadd** <groupe> | Ajout d'un groupe | |
| **groupdel** <groupe> | Suppression d'un groupe | |
| **groupmod** | Modification nom et GID | *groupmod –g 550 technique*<br>*groupmod –n technique tech* |
| **gpasswd** <groupe> | Modification d'un groupe<br>-a (ajout membre)<br>-d (supp membre)<br>-M (liste des membres)<br>-A (administrateur) | <br>*groupmod –a toto technique*<br>*groupmod –d titi technique*<br>*groupmod –M riri,fifi technique*<br>*groupmod –A riri technique* |

| | | |
|---|---|---|
| **useradd** <utilisateur> | Ajout d'un utilisateur<br>-g : groupe principal<br>-G : liste des groupes secondaires | *useradd toto –g technique –G direction,commercial* |
| **passwd** <utilisateur> | Changement mot de passe et verrouillage de compte<br>-S : état du compte<br>-l : verrouillage<br>-u : déverrouillage | |
| **userdel** <utilisateur> | Suppression d'un utilisateur<br>-r : efface le répertoire personnel | |
| **usermod** <utilisateur> | Modification utilisateur | *usermod toto –g direction –G technique* |

# EXERCICES :

Partie 2 Administration des utilisateurs

| |
|---|
| ▪ Utiliser la commande adduser pour créer quatre utilisateurs, util1, util2, util3, util4, spécifier à la demande le mot de passe toto, touche entrée pour toutes les autres informations et valider avec 'o'. |
| ▪ Afficher le contenu du répertoire /home, comment se nomment les nouveaux répertoires ? |
| ▪ Utiliser la commande groups pour afficher les groupes de chaque utilisateur, quel est le nom du groupe par défaut |
| ▪ Utiliser les commandes whoami et id, les informations affichées concernent quel compte ? |
| ▪ Utiliser la commande groupadd pour créer deux groupes gr1 et gr2. |
| ▪ Utiliser vi pour afficher le contenu du fichier /etc/passwd, que contient ce fichier. |
| ▪ Supprimer le x qui correspond au mot de passe des quatre utilisateurs (util1,util2, util3, util4), enregistrer avec vi |
| ▪ Définir avec la commande passwd un mot de passe pour util1 (ex: btsig2), ATTENTION spécifier bien le nom de l'utilisateur : passwd util1. Vérifier les changements dans /etc/passwd.<br>▪ Lancer une nouvelle console (Alt F2) et ouvrir une session avec le compte util1, mot de passe ?<br>▪ Lancer une autre console (Alt F3) et ouvrir une session avec le compte util2, mot de passe ?<br>▪ Revenir à la première console (Alt F1). |
| ▪ Ouvrir le fichier /etc/group avec vi et mettre les trois utilisateurs (util1, util3, util4) dans le groupe gr1 et util2 dans gr2, enregistrer. |
| ▪ Afficher les groupes de l'utilisateur util1, puis de l'utilisateur util2, avec la commande groups 38. Avec la commande su, ouvrir une session avec le nom d'utilisateur util1 |
| ▪ Tapez la commande cd et utiliser la commande pwd pour trouver dans quel répertoire vous êtes, lancer whoami et id |
| ▪ Revenir à la session root en tapant exit |
| ▪ Fermer la connexion root avec la commande logout et ouvrir une connexion avec util2. |
| ▪ Ouvrir une session root avec su, supprimer l'utilisateur util4 avec la commande userdel -r, vérifier dans le fichier /etc/passwd |
| ▪ Fermer la session root avec exit pour revenir à la connexion util2. |
| ▪ Créer dans /home/util2, trois fichiers (f1, f2 et f3) avec vi et un dossier nommé divers. |
| ▪ Fermer la connexion avec logout et ouvrir une nouvelle connexion avec root |

# LA GESTION DES DROITS

## Les droits de base :

Un fichier ou un répertoire appartiennent à un utilisateur propriétaire et à un groupe propriétaire.

**Groupes de droits :**

| | | |
|---|---|---|
| rwx | rwx | rwx |

1. r : lecture [read]
2. w : modification [write]

x : exécution [eXecute]

| **Droit** | **Fichier** | **répertoire** |
|---|---|---|
| 3. r | Autorise la lecture du contenu | Autorise à lister le contenu |
| 4. w | Autorise la modification | Autorise l'ajout et la suppression de sous-répertoires et fichiers |
| 5. x | Autorise l'exécution | Autorise le parcours (cd) |

Pour modifier les droits : la commande **chmod**

| | | | | |
|---|---|---|---|---|
| chmod | u | + | r | Fichier ou répertoire |
| | g | - | w | |
| | o | | x | |
| | a | | | |

u = user

g = group

o = other

a = all (u+g+o)

Autre possibilité : notation octale

| Valeurs octales | 4 | 2 | 1 | 4 | 2 | 1 | 4 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|
| Ensemble des droits | r | w | x | r | w | x | r | w | x |

Exemple : on veut affecter les droits : rwxr-xr-- au fichier 'fic1'

**chmod 754 fic1**

**Masque**

La commande 'mask' permet de définir les droits d'accès par défaut attribués aux nouveaux répertoires et fichiers.

La valeur par défaut du mask =

- 002 pour un utilisateur
- 022 pour root

Pour trouver les droits par défaut, il faut soustraire les droits du masque aux droits rwxrwxrwx (777)

*Exemple :* **umask 002**

| | Fichier ordinaire | | | | | | | | | Répertoire | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Valeurs octales | 4 | 2 | 1 | 4 | 2 | 1 | 4 | 2 | 1 | 4 | 2 | 1 | 4 | 2 | 1 | 4 | 2 | 1 |
| Ensemble des droits | r | w | x | r | w | X | r | w | x | r | w | x | r | w | x | r | w | x |
| Masque | - | - | - | - | - | - | - | w | - | - | - | - | - | - | - | - | w | - |
| Droits par défaut | r | w | - | r | w | - | r | - | - | r | w | x | r | w | x | r | - | x |

### a) les droits étendus : SUID, SGID et Sticky Bit

| Droit | Fichier | Répertoire | Notation | Commande |
|---|---|---|---|---|
| SUID | Execute le fichier sous l'identité du propriétaire | | x est remplacé par s ou S<br>s = s + x<br>S = s sans x | chmod 4000 |
| SGID | Execute le fichier sous l'identité du groupe | Les fichiers créés dans le répertoire héritent du groupe du répertoire à la place du groupe principal de l'utilisateur | Idem mais pour le groupe | chmod 2000 |
| Sticky | | Seul le propriétaire du fichier ou du répertoire peut supprimer les fichiers | x est remplacé par un t ou T au niveau des autres<br>t = t + x<br>T= t sans x | chmod 1000 |

### b) Les ACL (Access Control List)

| | |
|---|---|
| **getfacl** | Affiche la liste de contrôle d'accès |
| **setfacl** | Modifie la liste de contrôle d'accès |

*Exemples :*

*setfacl –s u::rwx, g::rwx,o::-,u:toto:rw fichier (définition complète)*

*setfacl –m u:toto:rwx fichier*

*setfacl –x u:toto fichier*

## Exercices

### Les droits d'accès

| |
|---|
| Avec la commande ls -l, afficher toutes les informations des fichiers de util2 (/home/util2) |
| Comment sont distingués les fichiers et les dossiers au niveau du premier caractère affiché ? |
| Que signifie le droit x pour un répertoire ? |
| Quels sont les droits par défaut pour le propriétaire, le groupe et les autres ? |
| Pour le fichier f1, mettre le groupe gr1, commande chgrp, afficher le résultat avec ls –l |
| Modifier les droits sur le fichier f1 pour autoriser tout pour le groupe gr1, commande chmod |
| Avec su, ouvrir une session avec util1, aller dans le répertoire /home/util2 et essayer de supprimer les fichiers f1 et f2 |
| Revenir à la connexion root avec exit, changer le propriétaire du fichier f2 (commande chown) pour mettre util1 et changer le groupe pour mettre gr2. |
| Changer les droits sur le fichier f2, propriétaire et groupe : lire seulement, les autres : aucun accès, le résultat avec ls -l |
| Créer un répertoire nommé test, dans le répertoire /home/rep1, mettre un fichier dans ce répertoire test |
| Faire le nécessaire pour que les utilisateurs (util1 à util3) ne voient pas les fichiers du répertoire /home/rep1, tout en ayant un accès en lecture sur les fichiers du répertoire test. |

### *La commande chmod avec une valeur octale*

*La valeur octale est la somme des valeurs correspondant aux droits que l'on souhaite fixer conformément au calcul suivant : r=4, w-2, x- l,*

*Par exemple : chmod 75l correspond à user: rwx, group : r - x, et others : -x.*

| |
|---|
| Modifier les droits de certains fichiers/répertoires avec la commande chmod <valeur octale>. |
| Supprimer tous les utilisateurs, les groupes et les répertoires créés dans ce Tp. |

# COMMANDES UTILES

## Afficher la date :

date

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/# date
vendredi 3 novembre 2017, 14:55:04 (UTC+0100)
root@debian:/#
```

## Afficher un calendrier :

**cal [mois] [année]**

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/# cal novembre 2017
   Novembre 2017
di lu ma me je ve sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30

root@debian:/#
```

ou

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/# cal 11 2017
   Novembre 2017
di lu ma me je ve sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30

root@debian:/#
```

## Lister les sessions ouvertes :

who

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/# who
jlb      tty7         2017-11-03 11:00 (:0)
root@debian:/#
```

Ou w

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/# w
 14:57:58 up  3:58,  1 user,  load average: 0,02, 0,01, 0,00
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
jlb      tty7     :0               11:00    3:58m 27.77s  0.17s x-session-manage
root@debian:/#
```

## Lister les logins ayant ouvert une session :

users

```
Fichier  Édition  Affichage  Recherche
root@debian:/# users
jlb
root@debian:/#
```

## Voir le répertoire courant :

pwd

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
jlb@debian:~$ pwd
/home/jlb
jlb@debian:~$
```

## Voir les informations utilisateur :

Id

## Nettoyer l'écran :

clear ou Ctrl+L

## Changement identité :

su -[nom utilisateur]

## Débloquer une console :

Il suffit de saisir reset.

## Eteindre la machine

# shutdown -h

## Rebooter la machine

# shutdown -r

# PROGRAMMATION DE SCRIPTS : UNE INTRODUCTION

Comment écrire de petits scripts permettant d'automatiser la réalisation de tâches répétitives.

## Introduction

Vous aurez envie d'écrire un script (petit programme écrit avec un langage simple : *shell*, *perl* ou autre) dès que vous aurez tapé dans un terminal quatre fois la même série de commandes et que vous vous apercevrez que vous êtes amené à le refaire de nombreuses fois.

Un script est une suite d'instructions élémentaires qui sont exécutées de façon séquentielle (les unes après les autres) par le langage de script. Dans cet article nous nous limiterons à l'utilisation du *shell* comme langage, et en particulier à du *shell* bash. En guise de première introduction, vous pouvez lire ce qui concerne les commandes du shell dans l'article Le Shell et les Commandes. Attention, n'espérez pas que le présent document constitue un manuel complet de programmation ! C'est une courte introduction qui nous l'espérons, vous permettra d'écrire de petits scripts qui vous rendront de précieux services.

## Notions de base

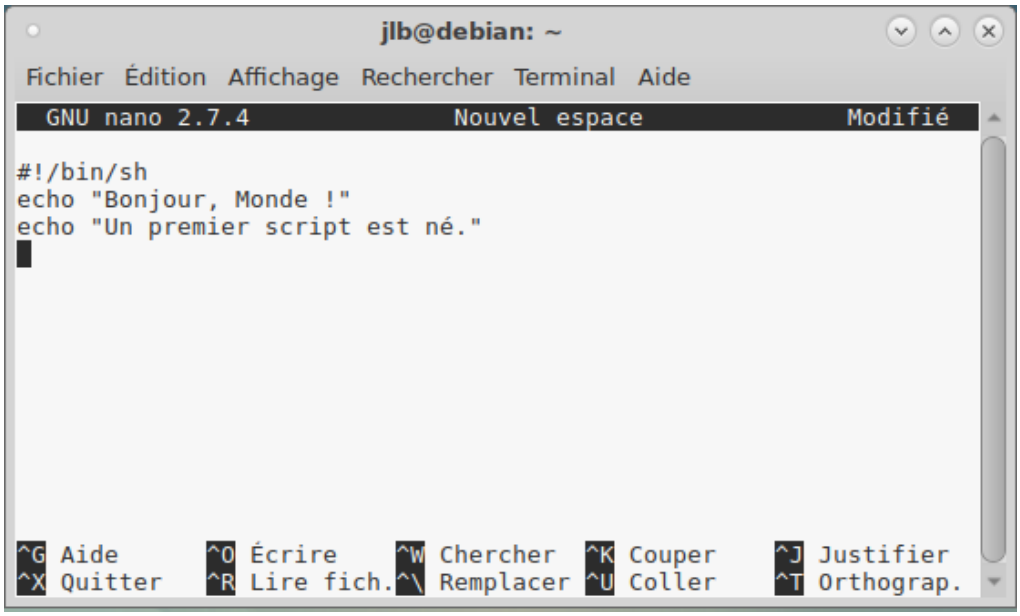
### Mon premier script

Pour commencer, il faut savoir qu'un script est un fichier texte standard pouvant être créé par n'importe quel éditeur : vi, emacs, kedit, Gedit,nano ou autre. D'autre part, conventionnellement, un script commence par une ligne de commentaire contenant le nom du langage à utiliser pour interpréter ce script, soit dans notre cas : /bin/sh (on parle alors de "script shell"). Donc un script shell élémentaire pourrait être :

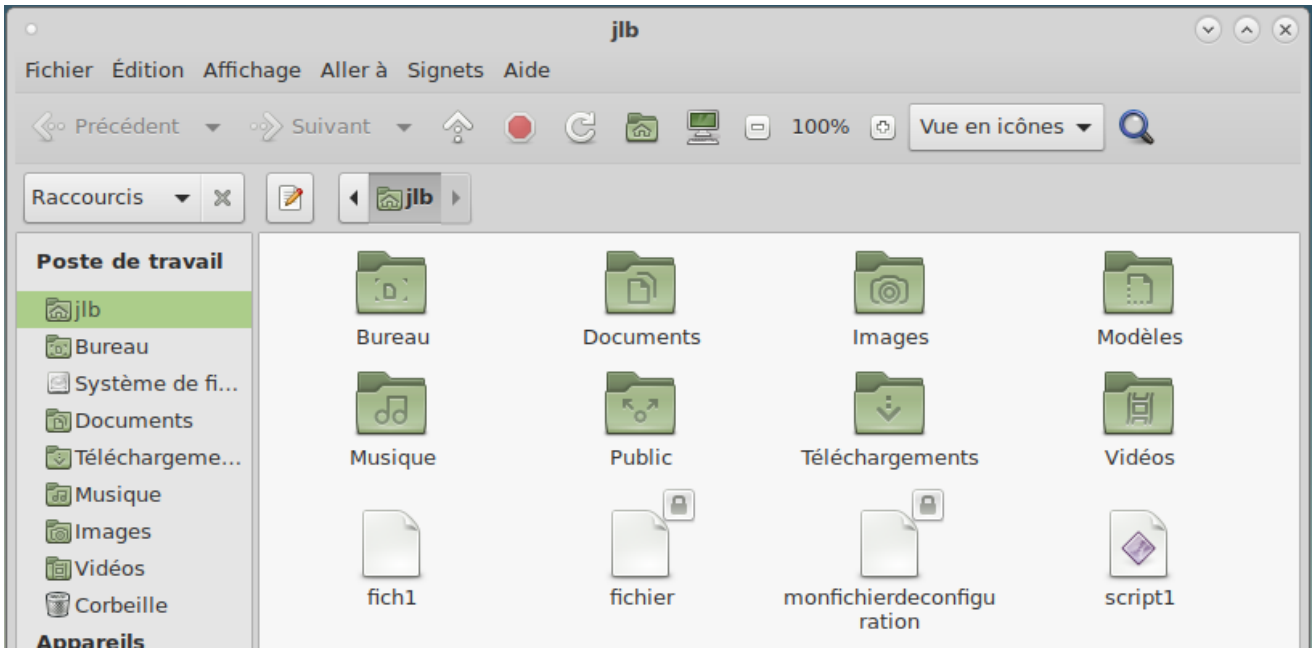
```
#!/bin/sh
```

**Note :** "#!" se prononce « *she bang* », soit « chi-bang ».

Évidemment un tel script ne fait rien ! Changeons cela. La commande qui affiche quelque chose à l'écran est echo. Donc pour créer le script bonjour_monde nous pouvons écrire :




Enregistrez le sous script1, il doit apparaitre dans votre dossier personnel




Comment on l'exécute ? C'est simple il suffit de taper :

Ce n'est pas super, vous auriez préféré taper quelque chose comme :

```
$ ./script1
Bonjour, Monde !
Un premier script est né.
```

C'est possible si vous avez au préalable rendu votre script exécutable par la commande :

```
$ chmod +x script1
$ ./bonjour_monde
Bonjour, Monde !
Un premier script est né.
```

Résumons : un script *shell* commence par : #!/bin/sh, il contient des commandes du shell et est rendu exécutable par chmod +x.

## QUELQUES CONSEILS CONCERNANT LES COMMENTAIRES

Dans un shell-script, est considéré comme un commentaire tout ce qui suit le caractère # et ce, jusqu'à la fin de la ligne. Usez et abusez des commentaires : lorsque vous relirez un script 6 mois après l'avoir écrit, vous serez bien content de l'avoir documenté. Un programme n'est jamais trop documenté, en revanche il peut

être mal documenté ! Un commentaire est bon lorsqu'il décrit pourquoi on fait quelque chose, pas quand il décrit ce que l'on fait.

Exemple :

```
#!/bin/sh
# pour i parcourant tous les fichiers,
for i in * ; do
# copier le fichier vers .bak
 cp $i $i.bak
# fin pour
done
```

Que fait le script ? Les commentaires ne l'expliquent pas ! Ce sont de mauvais commentaires.

En revanche :

```
#!/bin/sh
# on veut faire une copie de tous les fichiers
for i in * ; do
# sous le nom *.bak
 cp $i $i.bak
done
```

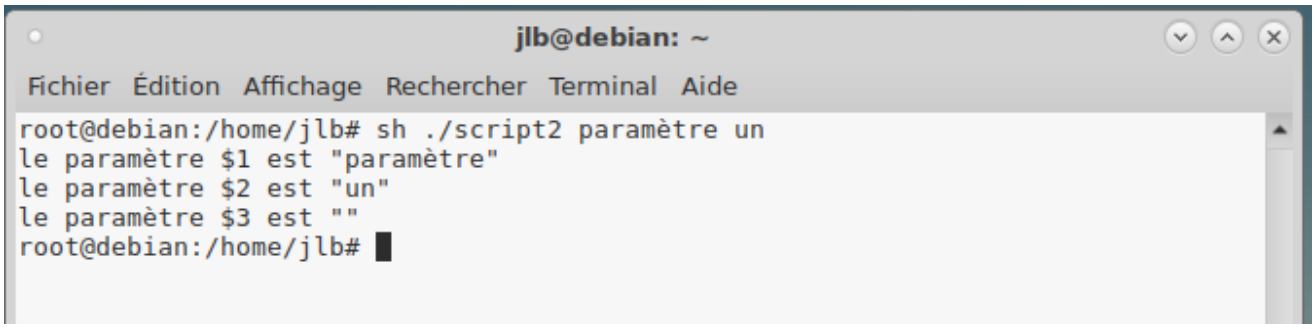
Là au moins, on sait ce qu'il se passe (il n'est pas encore important de connaître les commandes de ces deux fichiers).

## LE PASSAGE DE PARAMETRES

Un script ne sera, en général, que d'une utilisation marginale si vous ne pouvez pas modifier son comportement d'une manière ou d'une autre. On obtient cet effet en « passant » un (ou plusieurs) paramètre(s) au script *via* la ligne de commande. Voyons comment faire cela. Soit le script script02 :

```
#!/bin/sh
echo ''le paramètre'' \$1 est \"$1\"
echo ''le paramètre'' \$2 est \"$2\"
echo ''le paramètre'' \$3 est \"$3\"
```

Que fait-il ? Il affiche les uns après les autres les trois premiers paramètres du script, donc si l'on tape :

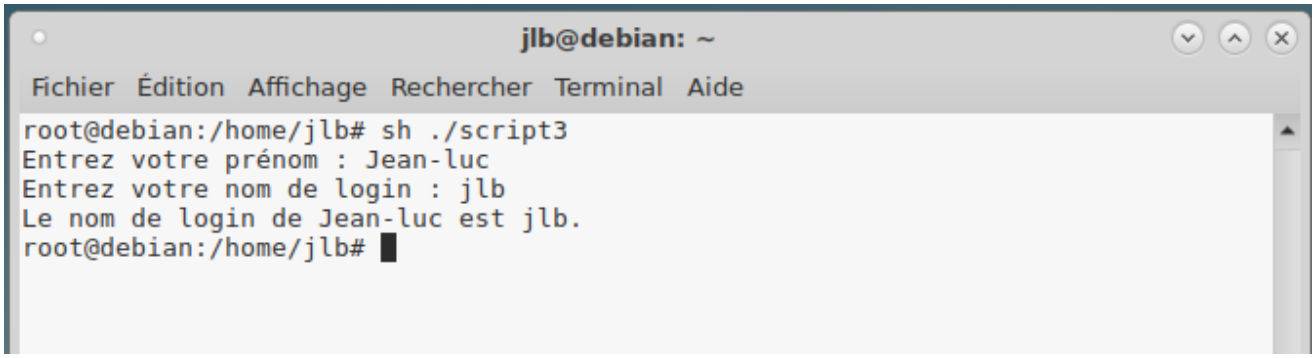



Donc, les variables $1, $2… $9 contiennent les « mots » numéro 1, 2… 9 de la ligne de commande. Attention : par « mot » on entend ensemble de caractères ne contenant pas de caractères de séparations. Les caractères de séparation sont l'espace, la tabulation, le retour chariot quand c'est possible et le point-virgule.

## SAISIR LA VALEUR D'UNE VARIABLE

Les paramètres permettent à l'utilisateur d'agir sur le déroulement du script avant son exécution. Mais il est aussi souvent intéressant de pouvoir agir sur le déroulement du script lors de son exécution, c'est ce que permet la commande : read nom_variable. Dans cette commande vous pouvez bien sûr remplacer nom_variable par le nom de variable qui vous convient le mieux. Voici un exemple simple.

```
#!/bin/sh
echo -n "Entrez votre prénom : "
read prenom
echo -n "Entrez votre nom de login : "
read nomlogin
echo "Le nom de login de $prenom est $nomlogin."
```




Lors du déroulement du script vous devez valider vos entrées en appuyant sur la touche "Entrée".

L'option -s de read permet de masquer la saisie. Par exemple read -s pass.

## Arithmétique

Vous vous doutez bien qu'il est possible de faire des calculs avec le shell. En fait, le shell ne "sait" faire que des calculs sur les nombres entiers (ceux qui n'ont pas de virgules ;-). Pour faire un calcul il faut encadrer celui-ci de : $(( un calcul )) ou $[ un calcul ]. Exemple, le script essai03 :

```
#!/bin/sh
echo 2+3*5 = $((2+3*5))
MACHIN=12
echo MACHIN*4 = $[$MACHIN*4]
Affichera :
$ sh essai03
2+3*5 = 17
MACHIN*4 = 48
```

Vous remarquerez que le shell respecte les priorités mathématiques habituelles (il fait les multiplications avant les additions !). L'opérateur puissance est "**" (ie : 2 puissance 5 s'écrit : 2**5). On peut utiliser des parenthèses pour modifier l'ordre des calculs.

## Les instructions de contrôle de scripts

Les instructions de contrôle du shell permettent de modifier l'exécution purement séquentielle d'un script. Jusqu'à maintenant, les scripts que nous avons créés n'étaient pas très complexes. Ils ne pouvaient de toute façon pas l'être car nous ne pouvions pas modifier l'ordre des instructions, ni en répéter.

## L'exécution conditionnelle

Lorsque vous programmerez des scripts, vous voudrez que vos scripts fassent une chose si une certaine condition est remplie et autre chose si elle ne l'est pas. La construction de bash qui permet cela est le fameux test : if then else fi. Sa syntaxe est la suivante (la partie else... est optionnelle) :

```
if <test> ;
then
  <instruction 1>
  <instruction 2>
  ...
  <instruction n>
else
  <instruction n+1>
  ...
  <instruction n+p>
fi
```

Il faut savoir que tous les programmes renvoient une valeur. Cette valeur est stockée dans la variable ? dont la valeur est, rappelons-le : "$?". Pour le shell une valeur nulle est synonyme de VRAI et une valeur non nulle est synonyme de FAUX. Ceci parce que, en général les programmes renvoie zéro quand tout c'est bien passé et un code d'erreur (nombre non nul) quand il s'en est produit une.

Il existe deux programmes particuliers : false et true. true renvoie toujours 0 et false renvoie toujours 1. Sachant cela, voyons ce que fait le programme suivant :

```
#!/bin/sh
if true ;
then
  echo "Le premier test est VRAI($?)"
else
  echo "le premier test est FAUX($?)"
fi

if false ;
then
  echo "Le second test est VRAI($?)"
else
  echo "Le second test est FAUX($?)"
fi
```

Affichera :

```
$ sh ./test
Le premier test est VRAI(0)
Le second test est FAUX(1)
$ _
```

On peut donc conclure que l'instruction if ... then ... else ... fi, fonctionne de la manière suivante : si (**if** en anglais) le test est VRAI(0) alors (**then** en anglais) le bloc d'instructions compris entre le then et le else (ou le fi en l'absence de else) est exécuté, sinon (**else** en anglais) le test est FAUX(différent de 0)) et on exécute le bloc d'instructions compris entre le else et le fi si ce bloc existe.

Bon, évidemment, des tests de cet ordre ne paraissent pas très utiles. Voyons maintenant de vrais tests.

## Les tests

Un test, nous l'avons vu, n'est rien de plus qu'une commande standard. Une des commandes standard est test, sa syntaxe est un peu complexe, je vais la décrire avec des exemples.

> si l'on veut tester l'existence d'un répertoire <machin>, on tapera : test -d <machin> ('d' comme ***d**irectory*)
>
> si l'on veut tester l'existence d'un fichier <machin>, on tapera : test -f <machin> ('f' comme ***f**ile*)
>
> si l'on veut tester l'existence d'un fichier ou répertoire <machin>, on tapera : test -e <machin> ('e' comme ***e**xist*)

Pour plus d'information faites : man test.

On peut aussi combiner deux tests par des opérations logiques : 'ou' correspond à -o ('o' comme ***o**r*), 'et' correspond à -a ('a' comme ***a**nd*) (à nouveau allez voir la man page), exemple :

test -x /bin/sh -a -d /etc

Cette instruction teste l'existence de l'exécutable /bin/sh (-x /bin/sh) et (-a) la présence d'un répertoire /etc (-d /etc).

On peut remplacer la commande test <un test> par [ <un test> ] qui est plus lisible, exemple :

```
if [ -x /bin/sh ] ; then
# ('x' comme  e"x"ecutable)
 echo /bin/sh est exécutable. C\'est bien.
else
 echo /bin/sh n\'est pas exécutable.
 echo Votre système n\'est pas normal.
fi
```

Toujours avec les crochets de test, si vous n'avez qu'une seule chose à faire en fonction du résultat d'un test, alors vous pouvez utiliser la syntaxe suivante :

[ -x /bin/sh ] && echo /bin/sh est exécutable.
ou encore :
[ -x /bin/sh ] || echo /bin/sh n\'est pas exécutable.

L'affichage du message est effectué, dans le premier cas que si le test est vrai et dans le second cas, que si le test est faux. Dans l'exemple on teste si /bin/sh est un fichier exécutable.
Cela allège le script sans pour autant le rendre illisible, si cette syntaxe est utilisée à bon escient.

Mais il n'y a pas que la commande test qui peut être employée. Par exemple, la commande grep renvoie 0 quand la recherche a réussi et 1 quand la recherche a échoué.

Par exemple :

```
if grep -E "Jean-Luc:" /etc/passwd > /dev/null ; then
  echo L\'utilisateur Jean-Luc existe.
else
  echo L'utilisateur Jean-Luc n\'existe pas.
fi
```

Cette série d'instruction teste la présence de l'utilisateur frederic dans le fichier /etc/passwd. Vous remarquerez que l'on a fait suivre la commande grep d'une redirection vers /dev/null pour que le résultat de cette commande ne soit pas affiché : c'est une utilisation classique. Ceci explique aussi l'expression : "Ils sont tellement intéressants, tes mails, que je les envoie vers /dev/null" ;-).

## Faire quelque chose de différent suivant la valeur d'une variable

L'instruction case ... esac permet de modifier le déroulement du script selon la valeur d'un paramètre ou d'une variable. On l'utilise le plus souvent quand les valeurs possibles sont en nombre restreint et peuvent être prévues. Les imprévus peuvent alors être représentés par le signe *. Demandons par exemple à l'utilisateur s'il souhaite afficher ou non les fichiers cachés du répertoire en cours.

```
#!/bin/sh
# pose la question et récupère la réponse
echo "Le contenu du répertoire courant va être affiché."
echo -n "Souhaitez-vous afficher aussi les fichiers cachés (oui/non) : "
read reponse
# agit selon la réponse
case $reponse in
  oui)
    clear
    ls -a;;
  non)
    ls;;
  *) echo "Erreur, vous deviez répondre par oui ou par non."
esac
```

Seules les réponses "oui" et "non" sont réellement attendues dans ce script, toute autre réponse engendrera le message d'erreur. On notera qu'ici l'écran est effacé avant l'affichage dans le cas d'une réponse positive, mais pas dans celui d'une réponse négative. Lorsque vous utilisez l'instruction case ... esac, faites bien attention de ne pas oublier les doubles points-virgules terminant les instructions de chacun des cas envisagés.

## Faire la même chose pour tous les éléments d'une liste

Lorsqu'on programme, on est souvent amené à faire la même chose **pour tous** les éléments d'une liste. Dans un shell script, il est bien évidemment possible de ne pas réécrire dix fois la même chose. On dira que l'on fait une boucle. L'instruction qui réalise une boucle est

```
for <variable> in <liste de valeurs pour la variable> ; do
  <instruction 1>
  ...
  <instruction n>
done
```

Voyons comment ça fonctionne. Supposons que nous souhaitions renommer tous nos fichiers *.tar.gz en *.tar.gz.old, nous taperons le script suivant :

```
#!/bin/sh
# x prend chacune des valeurs possibles correspondant
# au motif : *.tar.gz
for x in *.tar.gz ; do
  # tous les fichiers $x sont renommés $x.old
  echo "$x -> $x.old"
  mv $x $x.old
  # on finit notre boucle
done
```

Simple, non ? Un exemple plus complexe ? Supposons que nous voulions parcourir tous les sous-répertoires du répertoire courant pour faire cette même manipulation. Nous pourrions taper:

```
#!/bin/sh
for REP in `find -type d` ; do
  for FICH in $REP/*.tar.gz ; do
    if [ -f $FICH ] ; then
      mv $FICH $FICH.old
    else
      echo On ne renomme pas $FICH car ce n\'est pas un répertoire
    fi
  done
done
```

Explications : dans le premier 'for', on a précisé comme liste : `find -type d` (attention au sens des apostrophes, sur un clavier azerty français, on obtient ce symbole en appuyant sur ALTGR+é, ce ne sont pas des simples quotes ').
Lorsque l'on tape une commande entre apostrophes inverses, le shell exécute d'abord cette commande, et remplace l'expression entre apostrophes inverses par la sortie standard de cette commande (ce qu'elle affiche à l'écran). Donc, dans le cas qui nous intéresse, la liste est le résultat de la commande find -type d, c'est à dire la liste de tous les sous-répertoires du répertoire courant.
Ainsi, en ligne 2, on fait prendre à la variable REP le nom de chacun des sous-répertoires du répertoire courant, puis (en ligne 3) on fait prendre à la variable FICH le nom de chacun des fichiers .tar.gz de $REP (un des sous-répertoires), puis si $FICH est un fichier, on le renomme, sinon on affiche un avertissement.

Remarque : ce n'est pas le même fonctionnement que la boucle for d'autres langages (le pascal, le C ou le basic par exemple).

## Faire une même chose tant qu'une certaine condition est remplie

Pour faire une certaine chose **tant qu'**une condition est remplie, on utilise un autre type de boucle :

```
while <un test> ; do
  <instruction 1>
  ...
  <instruction n>
done
```

Supposons, par exemple que vous souhaitiez afficher les 100 premiers nombres (pour une obscure raison), alors vous taperez :

```
i=0
while [ $i -lt 100 ] ; do
  echo $i
  i=$[$i+1]
done
```

Remarque : -lt signifie "**l**esser **t**han" ou "plus petit que" (et -gt signifie "plus grand", ou "**g**reater **t**han").

Ici, on va afficher le contenu de i et lui ajouter 1 tant que i sera (-lt) plus petit que 100. Remarquez que 100 ne s'affiche pas, car -lt est "plus petit", mais pas "plus petit ou égal" (dans ce cas, utilisez-le et -ge pour "plus grand ou égal").

## Refaire à un autre endroit la même chose

Souvent, vous voudrez refaire ce que vous venez de taper autre part dans votre script. Dans ce cas il est inutile de retaper la même chose, préférez utiliser l'instruction function qui permet de réutiliser une portion de script (on dit : une "**fonction**"). Voyons un exemple :

```
#!/bin/sh
function addpath ()
{
  if echo $PATH | grep -v $1 >/dev/null; then
    PATH=$PATH:$1;
  fi;
  PATH=`echo $PATH | sed s/::/:/g`
}

addpath /opt/apps/bin
addpath /opt/office52/program
addpath /opt/gnome/bin
export PATH
```

Au début, nous avons défini une fonction nommée addpath dont le but est d'ajouter le premier argument ($1) de la fonction addpath à la variable PATH si ce premier argument n'est pas déjà présent (grep -v $1) dans la variable PATH, ainsi que supprimer les chemins vides (sed s/::/:/g) de PATH.
Ensuite, nous exécutons cette fonction pour trois arguments : /opt/apps/bin, /opt/office52/bin et /opt/gnome/bin.

En fait, une fonction est seulement un script écrit à l'intérieur d'un script. Les fonctions permettent surtout de ne pas multiplier les petits scripts, ainsi que de partager des variables sans se préoccuper de la clause export mais cela constitue une utilisation avancée du shell, nous n'irons pas plus loin dans cet article.

Remarque : le mot function peut être omis, mais son utilisation facilite la lecture du script.

## Autres types de répétitions.

Il existe d'autres types de répétitions, mais nous ne nous en occuperons pas dans cet article, je vous conseille la lecture, forcément profitable, de la "man page" de bash (man bash).

## Rendre un script exécutable

Pour exécuter un script, il faut que le fichier soit exécutable : chmod u+x nomscript.

Pour lancer un script, faire ./nomscript

Le script doit toujours commencer par #!/bin/bash

**Caractère particuliers :**

| Caractère | Signification |
|---|---|
| # | Commentaire |
| ; | Séparateur de commande |
| ;; | fin de liste de commande pour case |
| " | Banalisation partielle |
| ' | Banalisation totale |
| \ | Banalisation du caractère suivant |
| {} | Regroupe des commandes |

**Les arguments :**

Le passage d'argument se fait par nomscript arg1 arg2 arg3.

Les arguments ainsi passés ont pour nom $1, $1,$3.

$* désigne l'ensemble des arguments.

$# C'est le nombre d'arguments.

Affectation d'un argument : nomargument=valeurarguement

Code retour : 0 = succès de la commande.

**Exemple :**

Créer script à l'aide de vi ou autre contenant les lignes suivantes :

```
#!/usr/bin/env bash

echo "$$ : c'est le PID du script"
echo "$* : c'est la liste des arguments passé au script"
echo "$# : c'est le nombre d'argument passé à la commande"
echo "$1 : c'est le premier argument"
echo "$? : c'est le retour de l'écho (résultat commande) précédent (premier argument)"
echo "$0 : c'est le nom du script"
```

Rendre exécutable le script :

```
chmod u+x script
```

Lancer le script en lui passant 4 arguments :

```
./script 04 12 156 398
```

```
6066 : c'est le PID du script
04 12 156 398 : c'est la liste des arguments passé au script
4 : c'est le nombre d'argument passé à la commande
04 : c'est le premier argument
0 : c'est le retour de l'écho précédent (premier argument). 0 = réussite de la commande
./script : c'est le mon du script
```

# LE RESEAU

Comment connaitre l'ensemble des cartes réseaux installées sur votre ordinateur

Tapez la commande ip a dans le terminal

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
jlb@debian:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:8e:ca:9c brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.4/24 brd 192.168.0.255 scope global dynamic ens32
       valid_lft 40983sec preferred_lft 40983sec
    inet6 fe80::20c:29ff:fe8e:ca9c/64 scope link
       valid_lft forever preferred_lft forever
jlb@debian:~$
```

Lo correspond à l'adresse interne locale

Ens32 est la première carte réseau. Nous voyons que l'adresse est 192.168.0.4

Connaitre le serveur Dns

Tapez la commande nano /etc/resolv.conf dans le terminal

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
  GNU nano 2.7.4                    Fichier : /etc/resolv.conf

# Generated by NetworkManager
nameserver 192.168.0.254
```

Ici le serveur DNS est le 192.168.0.254

Connaitre le nom de la machine

Nano /etc/hostname

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
  GNU nano 2.7.4                    Fichier : /etc/hostname

debian
```

Comme vous le voyez l'ordinateur se nomme : debian

## La commande ss

SS (pour Socket Statistics) est un outil qui s'utilise en ligne de commande et qui vient remplacer l'ancien netstat aujourd'hui déprécié. SS permet en outre de récupérer un plus grand nombre d'informations que netstat, est plus rapide, et va chercher ces informations directement dans /proc.

### Utilisation basique : lister toutes les connexions

C'est évidemment la première chose que l'on a envie de taper…

Tapez la commande **ss**

```
jlb@debian: ~
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
root@debian:/home/jlb# ss
Netid State      Recv-Q Send-Q                                  Local Address:Port                                 Peer Address:Port
u_str ESTAB      0      0                                    /run/user/1000/bus 16882                                         * 16881
u_str ESTAB      0      0                                                     * 16787                                         * 16788
u_str ESTAB      0      0                                                     * 13687                                         * 13031
u_str ESTAB      0      0                            /run/systemd/journal/stdout 16044                                         * 16042
u_str ESTAB      0      0                                  @/tmp/dbus-yfVLiMlVK5 15298                                         * 16089
u_str ESTAB      0      0                                                     * 16197                                         * 16512
u_str ESTAB      0      0                       /var/run/dbus/system_bus_socket 14077                                         * 14076
u_str ESTAB      0      0                                                     * 12266                                         * 12267
u_str ESTAB      0      0                            /run/systemd/journal/stdout 17014                                         * 17626
u_str ESTAB      0      0                                    /run/user/1000/bus 15202                                         * 15986
u_str ESTAB      0      0                                                     * 15984                                         * 15198
u_str ESTAB      0      0                                                     * 14996                                         * 15751
u_str ESTAB      0      0                                  @/tmp/dbus-yfVLiMlVK5 17575                                         * 17574
u_str ESTAB      0      0                            /run/systemd/journal/stdout 17453                                         * 16719
u_str ESTAB      0      0                       /var/run/dbus/system_bus_socket 16512                                         * 16197
u_str ESTAB      0      0                                                     * 17108                                         * 17673
u_str ESTAB      0      0                                    @/tmp/.X11-unix/X0 17516                                         * 16780
u_str ESTAB      0      0                                    @/tmp/.X11-unix/X0 16676                                         * 17436
u_str ESTAB      0      0                                    /run/user/1000/bus 16091                                         * 16090
u_str ESTAB      0      0                                                     * 15248                                         * 15247
u_str ESTAB      0      0                            /run/systemd/journal/stdout 15160                                         * 15159
u_str ESTAB      0      0                                    /run/user/1000/bus 16879                                         * 17576
```

Mais comme vous avez pu le constater, un simple « ss » est un peu brutal puisqu'il nous sort TOUTES les connexions : aussi bien TCP, UDP, Socket Unix… Donc ça fait beaucoup. Heureusement on va pouvoir filtrer.

### Filtrer les connexions TCP, UDP ou Unix

Les options sont assez intuitives à utiliser :

- -t / --tcp : n'affiche que les connexions TCP
- -u / --udp : n'affiche que les connexions UDP
- -x / --unix : n'affiche que les connexions Unix

### Afficher les numéros de port / ne pas résoudre les IPs

Si vous souhaitez un affichage plus « numérique », vous pouvez demander à SS de ne pas résoudre noms de domaines et noms de services. Le traitement s'en trouvera allégé.

Commande ss -nt

### Afficher des statistiques avec SS

Et oui SS sait aussi faire ça ! Exemple : ss -s

```
root@debian:/home/jlb# ss -s
Total: 413 (kernel 416)
TCP:   0 (estab 0, closed 0, orphaned 0, synrecv 0, timewait 0/0), ports 0

Transport Total     IP        IPv6
*         416       -         -
RAW       1         0         1
UDP       5         3         2
TCP       0         0         0
INET      6         3         3
FRAG      0         0         0

root@debian:/home/jlb#
```

Pour en savoir plus visitez la page suivante :

https://www.guillaume-leduc.fr/outils-du-sysadmin-linux-ss.html